

八股文整理精华版本

目录

- java基础.....7
 - 说说你的JAVA是怎么学的？7
 - 目前最新的JDK版本是多少？ JDK重大更新的版本有哪些？ 有哪些重大更新？7
 - JDK和JRE的区别是什么？8
 - lamda表达式基本语法与底层实现？8
 - lamda中for和foreach的区别？8
 - hashCode()和equals()之间的关系.....8
 - JAVA中如何比较两个对象是否相等？ 为什么先比较hashCode(), 而不是先比较equals？8
 - 以“HashSet如何检查重复”为例子来说明为什么要有hashCode?.....9
 - 说说final、finally和finalize的区别？9
 - 为什么重写equals方法的同时，必须要重写hashCode方法？9
 - String，StringBuffer，StringBuilder的区别？9
 - 为什么String是不可变的？10
 - 泛型中extends和super的区别？10
 - 重载和重写的区别？10
 - Long的长度和范围?为什么要-1？10
 - List和set的区别？10
 - List，set，map的区别？10
 - 如何实现数组和List之间的转换？11
 - 多线程场景下如何使用ArrayList?.....11
 - ArrayList和LinkedList的区别?.....11
 - 生产场景下什么时候使用ArrayList，什么时候使用LinkedList？11
 - java中有哪些数据类型？11
 - Stringstr="donkey"与Stringstr=newString("donkey")一样吗？12
 - 抽象类是什么？12
 - 普通类和抽象类的区别是什么？12
 - 抽象类能使用final修饰吗？12
 - 接口是什么？12
 - 接口和抽象类的区别是什么？12
 - this和super在构造方法中的区别是什么？13
 - java异常的层次结构？13
 - java的集合类有了解吗？13
 - Collection和Collections的区别是什么？13
 - 进程和线程的区别是什么？13

守护线程是什么？	14
创建线程有哪几种方式？	14
callable和runnable的区别？	14
String属于基本的数据类型吗？	15
什么是自动拆箱和装箱？	15
Java中三个变量分别存储在哪块内存上？	15
java中的IO流分为几种？	15
throw和throws的区别？	15
finally一定会被执行吗？	16
常见的异常类有哪些？	16
JAVA容器都有哪些？	16
HashMap和Hashtable有什么区别？	16
为什么wait, notify, notifyall这些方法为什么不在thread里面？	16
Sleep(0)的作用.....	17
迭代器Iterator是什么？	17
线程的run()和start()有什么区别？	17
为什么我们调用start()方法时会执行run()方法，为什么我们不能直接调用run()方法？	17
面向对象与面向过程的区别？ 以及面向对象的三大特性？ 多态的条件？ ..	18
说说你对多态的理解.....	18
java中为什么不支持多继承？	18
static分配内存空间的时机.....	18
CHAR和VARCHAR的区别？	18
NOW()和CURRENT_DATE()有什么区别？	19
Static（静态）方法和普通方法之间的区别.....	19
为什么说Synchronized是非公平锁？	19
synchronized介绍下.....	19
synchronized和reentrantlock的区别？	19
lock和synchronized的区别？	20
谈一下CAS机制。	20
ABA问题.....	20
什么是java语言的跨平台性？ 原理是什么？	20
java与c++的区别？	21
floatf=3.4是否正确？	21
创建一个对象用什么关键字？ 对象实例与对象引用有何不同？	21
成员变量与局部变量的区别有哪些？	21
java集合中谁有序，谁无序？	22
java的反射机制是什么？ 有什么作用？	22
说说JAVA的类加载机制？	22
undolog的作用？ 在MVCC中有没有用到？	22
介绍一下MVCC.....	22
红黑树的特性？ 适用场景？	23
为什么hashmap使用红黑树而不是平衡二叉树？	24
ArrayLsit的底层？ 初始化过程和扩容时机.....	24

什么是接口幂等性？	24
在并发量特别高的情况下是使用synchronized还是使用reentrantLock？ ..	25
介绍下reentrantlock以及原理.....	25
线程的几种状态？	25
什么是序列化？	25
二叉树与平衡二叉树的区别？	26
上下文切换的过程？	26
内核态和用户态.....	26
sleep和wait有什么区别？	27
讲讲java的线程池？	27
线程池原理.....	27
线程池的几种状态？	27
线程池的参数.....	28
线程池默认工作流程.....	28
线程池配置无界队列了之后，拒绝策略怎么搞，什么时候用到无界队列.	28
为什么需要泛型？	29
从继承的角度看，将构造函数申明为私有会有什么作用？	29
HashMap的底层实现？ 他为什么是线程不安全的？ 想要线程安全有什么措施？	29
为什么concurrentHashMap是线程安全的？	29
ConcurrentHashMap和HashTable？	30
说一下ConcurrentHashMap中并发安全的实现.....	30
有哪些并发安全的实现方式.....	31
HashSet如何检查重复？	32
==和equals（）的区别？	32
Threadlocal的原理是什么？	32
new和malloc的区别？	32
Vector和list的区别？	33
Arraylist和vector区别？	33
创建对象的方法？	33
如何保证java线程的安全性？	33
抽象工厂模式和工厂模式的区别？	34
堆排序的基本思想.....	34
什么是平衡二叉树？	34
说说你知道的序列化方式.....	34
反射的作用？	34
反射是否破坏了控制范围这个目的？	34
java设计模式有哪些？	35
单例模式中懒汉模式和饿汉模式的区别.....	35
synchronized和volatile的区别？	35
语法糖.....	35
静态变量和实例变量的区别？	36
什么是集合？	36
集合与数组的区别是什么？	36

说说集合的框架（或：常用的集合类有哪些？）	36
Hashmap如何去重？	36
Mysql数据库.....	36
非关系型和关系型数据库的区别？	36
mysql与clickhouse.....	37
索引失效的情况.....	38
InnoDB索引与MyISAM索引实现的区别是什么？	38
说说分区索引.....	38
如果不加事务一个方法中有两条sql，第二条出现了问题，但是第一条正常执行，一般怎么办？	38
MyISAM和InnoDB分别是什么索引？ 聚集还是非聚集？	38
InnoDB为什么要用自增id作为主键？	39
说说分库与分表设计.....	39
聚集索引与非聚集索引的区别.....	39
如果查非聚簇索引，能不能不回表，什么情况不回表？	39
事务四大特性是什么？ 实现原理又是什么？	39
事务的并发会产生什么问题？ 事务隔离级别，每个级别会解决什么问题，MySQL默认是哪个级别？	40
Mysql的隔离级别是如何实现的？	40
Mysql如何优化？	40
不可重复读与幻读的区别是什么？	41
脏读和不可重复读的区别在于.....	41
讲一下Mysql主从复制？	41
mysql主从复制如何保证数据的一致性？	41
MySQL常见的存储引擎InnoDB、MyISAM的区别？	41
MySQL为什么使用B+树，而不是使用其他？ B+树的特点？	42
为什么说B+树比B树更适合实际应用中操作系统的文件索引和数据库索引？	42
B+树的特点是什么？	42
建立索引的使用场景.....	42
drop、delete与truncate的区别.....	42
Mysql中delete和drop的区别.....	43
什么叫视图？ 游标是什么？	43
什么是存储过程？ 用来调用什么？	43
内连接、自连接、外连接（左、右、全）、交叉连接的区别？	43
MVCC讲一下.....	44
什么是快照读和当前读？	44
limit的使用方法.....	44
count(*)、count(1)、count(col)区别？	44
Mysql中索引都有哪些分类？	45
Mysql和Oracle什么区别？	45
mysql主从复制.....	45
物理日志和逻辑日志的区别？	45
mysql三大日志——binlog、redolog、undolog.....	46

undolog的用途？	46
redolog和binlog的区别？	46
存储引擎MyISAM和InnoDB的区别？	46
mysql什么情况下索引会失效？（如果设置了合理的索引查询还是很慢可能是什么原因）	46
unionall和union的区间？	47
mysql中having和where的区别？	47
Mysql如何优化？	47
关系型数据库和非关系型数据库的区别？	47
主键索引与唯一性索引的区别.....	47
数据库常见的四种索引.....	48
mysql有哪些情况不适合建立索引.....	48
Mysql的执行引擎有哪些？有什么区别？适用于什么场景？	48
Mybatis.....	49
MyBatis是什么？	49
Mybatis优缺点.....	49
ORM是什么？	49
JDBC编程有哪些不足之处，Mybatis是如何解决的？	50
MyBatis编程步骤是什么样的？	50
Mybatis的功能架构是怎样的？	50
#{ }和\${ }的区别？	51
计算机网络.....	51
简述tcp和udp的区别？	51
为什么要三次握手？	51
为什么要四次挥手？	51
三次握手与四次挥手？	52
HTTP的头部信息包含哪些？	52
HTTP状态码.....	52
http请求的方式有哪些？	53
https和http的区别.....	53
说说对http的理解？	53
数据链路层、网络层、传输层、应用层、物理层的协议.....	53
OSI七层模型和TCP/IP五层模型.....	54
TCP的三次握手和四次挥手？	54
为什么TCP要建立三次握手？	55
说说cookie和session的区别.....	55
JVM.....	56
JVM内存区域总共分为两种类型.....	56
JVM中类加载机制.....	56
类加载器.....	56
什么是双亲委派机制？它有什么作用？	57
如何理解局部变量、成员变量以及静态变量？	57
G1和CMS收集器的区别.....	57
jvm中堆和栈的区别？	57

java对象的存储.....	58
垃圾回收.....	58
JVM组成部分?	59
JVM运行时数据区?	59
JMM是什么?	59
JVM常见命令.....	59
GC是什么? 为什么要有GC?	60
垃圾回收器的基本原理是什么? 垃圾回收器可以马上回收内存吗? 有什么办法主动通知虚拟机进行垃圾回收?	60
锁.....	60
乐观锁与悲观锁.....	60
公平锁与非公平锁.....	61
共享锁与独占锁.....	61
什么是可重入锁?	61
锁机制.....	61
什么是死锁?	61
产生死锁的条件.....	61
获取锁失败后当前线程会怎样.....	62
除了锁的方式, 还有哪些方式实现并发?	62
SSM.....	62
说说对springMVC的理解.....	62
@Data注解的作用?	63
@RequestBody注解的作用?	63
MVC模型介绍一下。.....	63
Spring三件套的框架?	63
AOP面向切面编程的好处? 以及怎么实现?	64
Springboot的优缺点?	64
spring事务相关的注解? 有哪些参数?	64
springboot和springmvc的区别.....	64
介绍下springmvc.....	65
springboot的常用注解?	65
Spring,springboot,springmvc都是干什么的?	65
Redis.....	66
Redis为什么会这么快?	66
介绍一下什么是阻塞I/O, 什么是非阻塞I/O, 什么是I/O多路复用?	66
redis的持久化机制?	66
引入缓存的好处?	66
如何提高缓存的利用率?	67
如何保证数据库和缓存中的数据一致性?	67
谈谈你对redis的理解?	67
分布式锁的三种实现方式是什么?	67
redis中的LUA脚本如何保证原子性的?	67
redis中的LUA脚本有什么用.....	68
redis集群的模式有哪些?	68

redis的淘汰机制?68

说说对缓存雪崩和缓存穿透、缓存击穿的理解.....69

缓存穿透与缓存击穿的区别?69

redis哨兵机制和集群有什么区别?69

RDB和AOF的实现原理和优缺点?70

Redis如何尽可能多的存储数据?70

redis的哈希表相对于redis的优势?70

数据结构.....71

说一个Java的一个排序方法?71

说一个有序链表增加查询效率的方法.....72

操作系统.....72

项目.....73

数据结构及算法.....77

常见的排序算法.....77

java基础

说说你的JAVA是怎么学的？

答：1）我在学这个技术栈的时候，首先观看了它的基础内容介绍视频，深刻了解它的基础框架与大致内容；

2）之后我会去github查询相关练手代码，我相信只有动手实践才能更好的掌握知识点；

3）在实践过程中，遇到问题我会有针对性的查阅相关解析博客或咨询学长学姐；

4）在成功解决问题后我还会记录下这些问题报错与所学所悟，我觉得如果未来工作中也遇到相关问题这样就可以很快的解决。

目前最新的JDK版本是多少？JDK重大更新的版本有哪些？有哪些重大更新？

（注意：！！由于更新换代很快，建议背的时候先查一查）

答：目前最新的JDK版本是JDK21。

重大更新的版本：

JDK5:JDK5是一个重要的里程碑，引入了许多重要的新功能，包括泛型、枚举、自动拆箱和装箱等。

JDK8：引入了Lambda表达式和函数式接口，提供了函数式编程的能力。JDK11：是一个长期支持版本，为java提供了许多改进和新功能。它包括模块化系统、动态类文件常量、局部变量类型判断等。

JDK14：引入了许多新的语言特性，例如switch表达式、文本块、箭头式操作符

等。

JDK17: 提供了许多新的功能和改进, 包括弃用一些过时的功能、垃圾收集器改进等

JDK和JRE的区别是什么?

答: JDK是JAVA开发工具包, 提供了JAVA的开发环境和运行环境。JRE是JAVA运行环境, 为JAVA的运行提供了所需环境。

总结:

JDK=JRE+多种java开发工具

JRE=JVM+各种类库

简单来说, 如果你需要运行java程序, 只需安装JRE就可以了, 如果你需要编写java程序, 就需要安装JDK。

lamda表达式基本语法与底层实现?

答: 是jdk8新引入的语法糖, 是一种用于取代匿名类, 把函数行为表述为函数式风格的一种匿名函数, 执行结果是函数式接口的匿名对象。它的底层还是用匿名内部类实现, 通过动态字节码技术实现;

lamda中for和foreach的区别?

答: 1) 普通for循环可以在各类型的循环操作中使用, 如果我们想遍历集合时还能对元素进行修改, 就要使用for循环;

2) foreach是对for循环的简化, 任何foreach循环都能改写成for循环, 但是反之则不行, 即foreach并不能完全替代for循环。foreach循环适用于对数组或实现了iterator接口的集合类进行遍历操作。

3) foreach适用于循环次数未知, 或者计算循环次数比较麻烦情况下, 但是一些更复杂的循环还是for循环效率更高。

hashCode()和equals()之间的关系

答: 1) 如果两个对象的hashCode不相同, 那么这两个对象肯定是不同的两个对象;

2) 如果两个对象的hashCode相同, 不代表这两个对象一定是同一个对象, 也可能是两个对象; 除非equals()返回true;

3) 如果两个对象相等, 那么他们的hashCode就一定相同

JAVA中如何比较两个对象是否相等? 为什么先比较hashCode(), 而不是先比较equals?

答: 在JAVA中比较两个对象是否相等时, 先调用对象的hashCode()方法得到哈

希值，如果哈希值不相同，就可以直接认为这两个对象不相同；如果哈希值相同，那么就会进一步调用equals()方法来最终确定两个对象是不是相等的，如果equals()返回true则相等，反之则不相等。

先比较hashCode是因为，equals方法的实现会比较繁琐，逻辑比较多；而hashCode()主要就是得到一个哈希值，实际上就是一个数字，相对而言比较简单，所以在比较两个对象时，通常都会先比较hashCode。

以“HashSet如何检查重复”为例子来说明为什么要有hashCode?

答：对象加入HashSet时，HashSet会先计算对象的hashCode值来判断对象加入的位置，看该位置是否有值，如果没有，HashSet会认为对象没有重复出现。

但是如果发现有值，这时会调用equals()方法来检查两个对象是否真的相同，如果两者相同，HashSet就不会让其加入操作成功。如果不相同的话会重新散列到其他位置。

先调用实现较为简单hashCode()，这样就大大减少了实现比较复杂的equals方法的次数，大大提高了响应执行速度。

说说final、finally和finalize的区别?

答：1) final：是一个修饰符，可以修饰变量、方法、引用和类；final修饰的类不能被继承，但是可以继承别的类；final修饰的方法可以被继承，但是不能重写；final修饰的变量为常量；final修饰的引用不能被修改，但是引用的内容可以改变；2) finally：是java中的一种异常处理机制，用在try,catch之后；并且附带一个语句块，这段语句不管有没有抛出异常，最终一定会被执行，经常被用在需要释放资源的情况下；

3) finalize：当垃圾回收器确定再无任何引用指向某个对象实例的时候，就会在销毁对象之前调用finalize()方法，一般用于清理对象所占用的空间。

为什么重写equals方法的同时，必须要重写hashCode方法?

答：当我们将equals方法重写后有必要将hashCode方法也重写，这样做才能保证不违背hashCode方法中“相同对象必须有相同哈希值”的约定，也就是必须保证重写后的equals方法认定相同的两个对象拥有相同的哈希值。

两个对象的hashCode()相同，则equals()也一定为true正确吗?

答：不正确，两个对象的hashCode()相同，equals()不一定true，hashCode()相等只能说明这两个键值对的哈希值相同，不代表这两个键值对相等。

String, StringBuffer, StringBuilder的区别?

答：1) String是字符串常量，是不可变的，如果尝试去修改，会新生成一个字符串对象，StringBuffer和StringBuilder是字符串变量，是可变的；

2) StringBuffer是线程安全的，StringBuilder是线程不安全的，但是StringBuilder的性能却高于StringBuffer，所以在单线程环境下推荐使用StrinBuilder，效率会

更高，多线程环境下推荐使用StringBuffer。

为什么String是不可变的？

答:String不可变的真正原因是:保存字符串的数组被final修饰且为私有的,并且String类没有提供修改这个字符串的方法

泛型中extends和super的区别？

答：1) <?extends T>表示包括T在内的任何T的子类；2) <?super T>表示包括T在内的任何T的父类

重载和重写的区别？

答：重载（overload）：在一个类中，同名的方法如果有不同的参数列表（比如参数类型不同、参数个数不同）则视为重载。

重写（override）：从字面上看，重写就是重新写一遍的意思，其实就是在子类中把父类本身有的方法重新写一遍。子类继承了父类的方法，但有时子类并不想原封不动的继承父类中的某个方法，所以在方法名，参数列表，返回类型都相同的情况下，对方法体进行修改，这就是重写。但是要注意子类方法的访问修饰权限不能小于父类的。

Long的长度和范围?为什么要-1?

答：长度是8个字节，64位。范围是 -2^{63} - 2^{63} -1。减1是因为java中的long类型是使用有符号数表示的。对于有符号数，最高位用于表示正负符号，0表示正数，1表示负数。因此，对于64位的long类型，最高位是符号位，剩下的63位用于表示数值的大小。由于要考虑正数和负数的表示，所以最大值是 2^{63} -1，而不是 2^{63} 。

List和set的区别？

答：1) List有序可重复，set无序不可重复；
2) List允许多个Null元素对象,set最多允许有一个Null元素对象；
3) List可以使用Iterator取出所有元素，再逐一遍历，还可以使用get(int index)获取指定下标的元素;set取元素时只能用Iterator接口取得所有元素，再逐一遍历各个元素。

List, set, map的区别？

答：List：有序可重复，允许有多个null值对象；常见的实现类有：ArrayList，LinkedList，Vector等；

Set: 无序不可重复，只允许最多有一个null元素。常见的实现类有：HashSet、TreeSet等；

Map:是一个键值对集合，存储键、值和之间的映射，key无序，唯一；value不要求有序，允许重复。从map集合中检索元素时，只要给出键对象，就会返回对应的值对象；Map常用的实现类：HashMap、TreeMap等

如何实现数组和List之间的转换？

答：1) 数组转List:使用Arrays.asList(array)进行存储；2)
List转数组：使用List自带的toArray（）方法。

多线程场景下如何使用ArrayList？

答：ArrayList不是线程安全的，如果遇到多线程场景，可以通过Collections的synchronizedList方法将其转换为线程安全的容器后再使用；

ArrayList和LinkedList的区别？

答：1) 首先，他们的底层数据结构不同，ArrayList底层是基于动态数组实现的，连续内存存储。LinkedList底层是基于链表实现的，可以存储在分散的内存中；2)
由于底层数据结构不同，他们所适用的场景也不同，ArrayList更适合随机查找；LinkedList更适合插入和删除操作，不适合查询；

生产场景下什么时候使用ArrayList，什么时候使用LinkedList？

答：ArrayList：底层使用动态数组实现的，连续内存存储，它在随机访问时有很好的性能，时间复杂度为 $O(1)$ ，但是插入和删除时性能较差，因为需要移动元素，时间复杂度为 $O(n)$ 。所以，如果你的需求的是大量的随机访问，少量的插入和删除，那么ArrayList就是一个很好的选择；

LinkedList：底层是使用双向链表实现的，存储在分散的内存中，它在插入和删除元素时性能很好，时间复杂度是 $O(1)$ ，但是在随机访问时性能较差，因为需要从头部或者尾部开始遍历，时间复杂度为 $O(n)$ 。所以，如果你的需求是大量的插入和删除操作，少量的随机访问操作，那么LinkedList是一个更好的选择。

java中有哪些数据类型？

答：java中数据类型分为基本数据类型和引用数据类型。

- 1) 基本类型有8种：byte（1字节）、short（2字节）、int（4字节）、long（8字节）、float（4字节）、double（8字节）、char（2字节）、boolean（默认值false）。
- 2) 引用类型：类（默认值null）、接口（默认值null）、数组（默认值null）

Stringstr="donkey"与Stringstr=newString("donkey")一样吗?

答：不一样，因为内存的分配方式不一样。

Stringstr="donkey"，java虚拟机会将其分配到常量池中；而Stringstr=new String("donkey")则会被分配到堆内存中。

抽象类是什么?

答：拥有抽象方法（指没有方法体的方法，同时抽象方法必须使用关键字abstract做修饰）的类就是抽象类，抽象类要使用abstract关键字声明。

普通类和抽象类的区别是什么?

答：1) 普通类不能包含抽象方法，抽象类可以有抽象方法；有抽象方法的类一定是抽象类；

2) 普通类可以直接实例化，抽象类不能被实例化。

3) 抽象类的子类必须实现抽象类中的所有抽象方法，否则子类仍然是抽象类；

4) 抽象方法不能声明为静态，不能被static、final修饰。

抽象类能使用final修饰吗?

答：不能，定义抽象类就是让其它类继承的，如果定义为final该类就不能被继承，这样彼此就会产生矛盾，所以final不能修饰抽象类。

接口是什么?

答：接口可以理解为一种特殊的类，里面全部是由全局常量和公共的抽象方法所组成，接口是解决java无法使用多继承的一种手段，但是接口在实际中更多的作用是制定标准。

接口和抽象类的区别是什么?

答：1) 接口必须使用implements来实现；抽象类的子类使用extends来继承；2) 接口不能有构造函数；抽象类可以有构造函数。

3) 接口没有普通成员变量；抽象类可以有普通成员变量；

4) 一个类只能继承一个抽象类，但可以实现多个接口；

5) 抽象类中成员变量默认default，可在子类中被重新定义，也可以被重新赋值。

抽象方法被abstract修饰，不能被private、static等修饰；接口中成员变量默认为publicstaticfinal修饰，必须赋初值，不能被修改，其所有的成员方法默认使用publicabstract修饰。

this和super在构造方法中的区别是什么？

- 答：1) this指向本对象的指针，super是一个关键字；
2) super从子类调用父类构造，this在同一个类中调用其他构造，均需要放在第一行，否则编译不予通过；
3) this和super不能出现在同一个构造器中，否则编译不予通过；

java异常的层次结构？

答：在java中，异常是一种用于处理程序执行过程中出现的错误或异常情况的机制。JAVA异常的层次结构如下：

- 1) java.lang.Throwable，是所有异常的父类，它有两个子类：Exception和Error。2) java.lang.Exception：表示程序可以捕获和处理的异常。Exception又分为两种类型：Checkedexception-可检查的异常-它是在编译时强制要求程序必须处理的异常；Uncheckedexception-非检查的异常，也称为运行时异常。这些异常在编译时不强制要求程序处理。
3) java.lang.error：表示严重的错误，通常无法恢复。

java的集合类有了解吗？

答：常见的集合类有这些：

- 1) ArrayList：实现了动态数组，可根据需要自动扩容
- 2) LinkedList：基于链表实现的列表，提供了高效的插入和删除操作。
- 3) HashSet：哈希表实现的集合，不允许重复的元素；
- 4) TreeSet：基于红黑树实现的有序集合，它会按照自然排序或自定义的排序规则对元素进行排序；
- 5) HashMap：基于哈希表实现的键值对映射；
- 6) TreeMap:基于红黑树实现的有序键值对的映射；
- 7) LinkedHashMap：基于哈希表和双向链表实现的有序键值对映射；

Collection和Collections的区别是什么？

- 答：1)Collection是最基本的集合接口，它派生了两个子接口list和set，分别定义了两种不同的存储方式；
2) Collections是一个包装类，它包含了各种有关集合操作的静态方法（对集合的搜索、排序、线程安全化等）。此类不能实例化，就像一个工具类，服务于Collection框架。

进程和线程的区别是什么？

答：进程和线程的主要差别在于他们是操作系统不同的资源管理方式。进程有独

立的地址空间，一个进程崩溃后，在保护模式下不会对其他进程产生影响，而线程只是一个进程中的不同执行路径。

线程之间没有独立的地址空间，一个线程死掉就等于整个进程死掉，所以多进程的程序要比多线程的程序健壮，但在进程切换时，耗费资源比较大，效率要差一些。

守护线程是什么？

答：守护线程，是个服务线程，准确的来说就是服务其他的线程，它能够自我结束。如果JVM中没有一个正在运行的非守护线程，这个时候，JVM会退出。JVM中的垃圾回收线程就是典型的守护线程，如果说没有守护线程，JVM就永远不会退出了。

创建线程有哪几种方式？

答：1) 继承**Thread**类创建进程；

这是创建线程的最基本的方法，我们可以创建一个新的类，继承自**Thread**类，然后重写其**run()**方法，将我们的任务代码写在**run()**方法中，然后创建该类的对象并调用其**start()**方法来启动线程；

2) 实现**Runnable**接口；

我们可以创建一个新的类，实现**Runnable**接口，然后将我们的任务代码写在**run()**方法中。然后创建该类的对象，将其作为参数传递给**Thread**类的构造器，创建**Thread**类的对象并调用其**start()**方法来启动线程。

3) 通过**Callable**接口和**FutureTask**类

Callable接口与**Runnable**接口类似，但是它可以返回一个结果，或者抛出一个异常。我们可以创建一个新的类，实现**Callable**接口，然后将我们的任务代码写在**call()**方法中。然后创建该类的对象，将其作为参数传递给**FutureTask**类的构造器，创建**FutureTask**类的对象。最后，将**FutureTask**类的对象作为参数传递给**Thread**类的构造器，创建**Thread**类的对象并调用其**start()**方法来启动线程。

4) 使用线程池

JAVA提供了线程池API，我们可以通过**Executors**类的一些静态工厂方法来创建线程池，然后调用其**execute()**或**submit()**方法来启动线程。线程池可以有效的管理和控制线程，避免大量线程的创建和销毁带来的性能开销。

以上四种方式，前两种是最基本的创建线程的方式，但是在实际开发中，我们通常会选择线程池，因为他可以提供更好的性能和更易于管理的线程的生命周期。

callable和runnable的区别？

答：1) **Runnable**接口只有一个**run()**方法，该方法不返回任何值，也无法对**run()**方法抛出的异常进行任何处理；**Callable**接口则有一个**call()**方法，它可以返回一个值，并且可以对异常进行捕获和处理

2) 使用场景不同

Runnable适用于那些不需要返回值，且不会抛出**checkedException**的情况，比如简单的打印输出或者修改一些共享的变量。

Callable适用于那些需要返回值或者需要抛出**checkedException**的情况，比如对某个任务的计算结果进行处理。在**Java**中，常常使用**Callable**来实现异步任务的处理，以提高系统的吞吐量和响应速度。

String属于基本的数据类型吗？

答：不属于。

八种基本数据类型：**byte**（1字节）、**short**（2字节）、**char**（2字节）、**int**（4字节）、**long**（8字节）、**double**（8字节）、**float**（4字节）、**boolean**（没有明确规范）

存储位置

基本数据类型存储在栈中，引用数据类型存储在堆中

如何将字符串反转？

答：将对象封装到**stringBuilder**中，调用**reverse**方法反转。

什么是自动拆箱和装箱？

答：自动装箱就是将基本数据类型自动转换成对应的包装类；
自动拆箱就是将包装类自动转换成对应的基本数据类型；

Java中三个变量分别存储在哪块内存上？

答：局部变量：存储在栈内存中；

实例变量：存储在堆内存中；

静态变量：存储在方法区中

java中的IO流分为几种？

答：1）按流划分，可以分为输入流和输出流；

2）按单位划分，可以分为字节流和字符流；

字节流：**InputStream,OutputStream**

字符流：**Reader,Writer**

throw和throws的区别？

答：1）**throw**

作用在方法内，表示抛出具体的异常，由方法体内的语句处理；
一定抛出了异常；

2）**throws**

作用在方法的声明上，表示可能抛出的异常，由调用者来进行异常的处理；

可能出现异常，不一定会发生异常。

try-catch-finally中，如果**catch**中**return**了，**finally**还会执行吗？答：
会执行，执行顺序是：catch->finally->catch中return

finally一定会被执行吗？

答：除非发生system.exit()调用、死循环、线程中断等特殊情况，finally块通常会执行。

常见的异常类有哪些？

答：1) NullPointerException: 空指针异常；
2) SQLException: 数据库相关的异常；
3) IndexOutOfBoundsException: 数组下角标越界异常；
4) FileNotFoundException: 打开文件失败时抛出；
5) IOException: 当发生某种IO异常时抛出；

JAVA容器都有哪些？

答：1) Collection
set: HashSet、TreeSet
list: ArrayList、LinkedList、Vector
2) Map
HashMap、HashTable、TreeMap

HashMap和Hashtable有什么区别？

答：1) HashMap是线程不安全的，HashTable是线程安全的
2) HashMap中允许键和值为null, HashTable不允许；
3) HashMap的默认容器是16，为2倍扩容，HashTable默认是11，为2倍+1扩容。
4) HashMap在数组+链表的结构中引入了红黑树，Hashtable没有

为什么wait, notify, notifyall这些方法为什么不在thread里面？

答：1) wait()、notify()和notifyAll()方法需要在拥有锁的情况下调用，以确保对锁的正确使用。由于锁是对象级别的，因此将这些方法定义在Object类中更合适。

2) 由于多线程编程需要在共享对象上进行协作和通信，而不仅仅局限于Thread类，因此将这些方法定义在Object类中可以应用于任何Java对象。因此，通过将wait()、notify()和notifyAll()方法定义在Object类中，Java提供了一种通用且灵活的方式来实现线程之间的协作与通信。在实际的多线程编程中，您可以使用任何对象作为锁，并在该对象上使用这些方法来实现所需的线程协作机制。

Sleep(0)的作用

答：调用sleep方法可以暂停当前线程的执行，让其他线程有机会执行；Sleep(0)是一种特殊的用法，它的作用是让当前线程放弃剩余的时间片，并将CPU时间让给其他具有相同优先级的线程。换句话说，sleep(0)可以让当前线程主动让出CPU资源，以便其他线程可以获得执行的机会；

迭代器Iterator是什么？

答：为了方便的处理集合中的元素，Java中出现了一个对象，该对象提供了一些方法专门处理集合中的元素，例如删除和获取集合中的元素，该对象就叫做迭代器。

线程的run()和start()有什么区别？

答：1) start()方法用于启动线程，run()方法用于执行线程的运行时代码。
2) run()可以重复调用，而start()只能调用一次。
3) 直接调用run()方法，会像普通方法一样在当前线程中顺序执行run()方法的内容，这并不会启动一个新的线程；调用start()方法会创建一个新的线程，并在新的线程中并行执行run方法的内容
因此，在java多线程编程中，我们应该始终使用start()方法来启动新线程，而不是直接调用run()方法。

为什么我们调用start()方法时会执行run()方法，为什么我们不能直接调用run()方法？

答：new一个Thread，线程进入了新建状态。调用start()方法，会启动一个线程并使线程进入了就绪状态，当分配到时间片后就可以开始运行了。start()会执行线程的相应准备工作，然后自动执行run()方法的内容，这是真正的多线程工作。而直接执行run()方法，会把run方法当成一个普通方法去执行，并不会在某个线程中执行它，所以这并不是多线程工作。

总结：调用start方法方可启动线程并使线程进入就绪状态，而run方法只是thread的一个普通方法调用，还是在主线程里执行。

面向对象与面向过程的区别？以及面向对象的三大特性？多态的条件？

答：它们是两种不同的处理问题的角度。面向过程更注重事情的每一个步骤及顺序，面向对象更注重事情有哪些参与者（对象）、及各自需要做什么。面向过程比较直接高效，而面向对象更易于复用、扩展和维护。

三大特性：封装、继承和多态

封装：封装的意义，在于明确标识出允许外部使用的所有成员函数和数据项。

内部细节对外部调用透明，外部调用无需修改或者关心内部实现。

继承：继承父类的方法，并作出自己的改变或扩展。子类共性的方法或者属性直接使用父类的，而不需要自己再定义，只需扩展自己个性化的。

多态：指的是在不同的对象上调用同一个方法时，可以产生不同的行为。多态的作用是提高了代码的可维护性和可扩展性，降低了代码的耦合度。

多态的三个条件：继承、方法重写、父类引用指向子类对象

说说你对多态的理解

答：多态是封装、继承之后，面向对象的第三大特性。多态是同一个行为具有不同的表现形式或形态的能力；同一方法可以根据发送对象的不同而采用不同的行为方式。多态的作用是提高了代码的可维护性和可扩展性，降低了代码的耦合度。

多态必须要满足三个条件，分别是继承、方法重写、父类引用指向子类对象

java中为什么不支持多继承？

答：为了降低复杂度和简化语言，java中不支持多重继承。考虑一个场景，其中A、B和C是三个类。C类继承A和B类。如果A类和B类有相同的方法，并且你从子类对象中调用它，那么调用A类或B类的方法就会有歧义。

static分配内存空间的时机

答：static变量在项目启动时就分配内存，直到程序结束释放内存；static方法只有在调用的时候才开始分配内存

CHAR和VARCHAR的区别？

答：1) char的长度是不可变的，用空格填充到指定长度大小，而varchar的长度是可变的；

2) char的存取速度比varchar快的多；

3) char的存储方式是：对英文字符占用1个字节，对一个汉字占用两个字节。

varchar的存储方式是：对每个英文字符和汉字都占用2个字节。

NOW()和CURRENT_DATE()有什么区别？

答：NOW()命令用于显示当前年份，月份，日期，小时，分钟和秒；CURRENT_DATE()仅显示当前年份，月份和日期。

Static（静态）方法和普通方法之间的区别

- 1.静态方法在创建对象前就可以使用了，非静态方法必须通过new出来的对象调用。
- 2.静态方法可以通过类名::方法名直接调用。普通方法需要创建一个实例，也就是new一个对象，然后通过对象名->方法名的方式来调用
- 3.静态类只能包含静态成员，否则会抛出编译错误;然而非静态类既可以包含非静态成员也可以包含静态成员
- 4.静态类是不能实例化,之所以不能实例化,是因为静态类会导致C#编译器将该类同时标记为abstract和sealed,并且编译器不会在类型中
- 5.生成一个实例的构造函数，从而导致静态类不能实例化;非静态类可以，并且静态成员的访问只能通过类来进行访问，因为静态成员是属于类的。

为什么说synchronized是非公平锁？

答：当锁被释放后，任何一个线程都有机会竞争得到锁，这样做的目的是提高效率，但是缺点是可能产生线程饥饿现象。

synchronized介绍下

答：synchronized关键字解决的是多个线程之间访问资源的同步性，他翻译成中文是同步的意思，也称之为同步锁；

synchronized的作用是保证是同一时刻，被修饰的代码块或者方法只会有一个线程执行，以达到保证并发安全的效果。

synchronized和reentrantlock的区别？

答：synchronized（内置锁），ReentrantLock（可重入锁）

- 1) 用法不同：synchronized可用来修饰普通方法、静态方法和代码块，而ReentrantLock只能用在代码块上；
- 2) 获取锁和释放锁的方式不同：synchronized会自动加锁和释放锁，当进入synchronized修饰的代码块后会自动加锁，当离开synchronized的代码块之后会自动释放锁；而ReentrantLock需要手动加锁和释放锁；
- 3) 锁类型不同：synchronized属于非公平锁，而ReentrantLock既可以是公平锁也可以是非公平锁；
- 4) 响应中断不同：如果发生了死锁，使用synchronized会一直等待下去；而使用ReentrantLock可以响应中断并释放锁，从而解决死锁问题。

5) 底层实现不同

synchronized是JVM层面通过监视器实现的，而ReentrantLock是通过AQS程序级别的API实现的。

lock和synchronized的区别？

答：1) synchronized是关键字，Lock是接口；

2) synchronized是隐式的加锁，lock是显示的加锁；

3) Synchronized底层采用的是objectMonitor，lock采用的是AQS；4)

synchronized是阻塞式加锁，lock是非阻塞式加锁

5) synchronized只支持非公平锁，lock支持非公平锁和公平锁；

谈一下CAS机制。

CAS它的全称是叫CompareAndSwap，比较并交换的意思，它的主要功能是保证在多线程的环境下，对于共享变量修改的一个原子性。是乐观锁的一种实现形式。

CAS操作包含三个操作数——V：内存位置、A：预期原值、B：新值。如果内存位置的值与预期原值相匹配，那么处理器会自动将该位置的值更新为新值。

可能会存在ABA问题，可通过每次修改设置不同的版本号来解决。

ABA问题

答：因为CAS需要在操作值的时候检查下值有没有发生变化，如果没有发生变化则更新，但是如果一个值原来是A，变成了B，又变成了A，那么使用CAS进行检查时会发现它的值没有发生变化，但是实际上却发生变化了。

大驼峰与小驼峰有什么区别？

根本区别就在于首单词的首字母是否大写，若是大写，则是大驼峰命名法；

若是小写，则是小驼峰命名法。

什么是java语言的跨平台性？原理是什么？

答：所谓跨平台性，是指java语言编写的程序，一次编译后，可以在多个系统平台上运行；

实现原理：Java程序是通过java虚拟机在系统平台上运行的，只要该系统安装相应的java虚拟机，就可以运行java程序。

java与c++的区别？

答：（java有着c++语言中的众多优点，同时摒弃了c++语言中难以理解的多继承、指针部分）

- 1) 都是面向对象的语言，都支持封装、继承和多态；
- 2) java不提供指针来直接访问内存，程序内存更加安全
- 3) java的类是单继承的，C++支持多继承；虽然java的类不可以多继承，但是接口可以多继承
- 4) java有自动内存管理机制，不需要程序员手动释放无用内存。

floatf=3.4是否正确？

答：不正确。3.4是双精度数，将双精度型（double）赋值给浮点型（float）属于向下转型，会造成精度损失，因此需要强制类型转换floatf=(float)3.4;或者写成floatf=3.4F。

创建一个对象用什么关键字？对象实例与对象引用有何不同？

答：new关键字，new创建对象实例（对象实例在堆内存中），对象引用指向对象实例（对象引用存放在栈内存中）。一个对象引用可以指向0或切割对象，一个对象可以有n个引用指向它。

成员变量与局部变量的区别有哪些？

答：成员变量：方法外部，类内部定义的变量；

局部变量：类的方法中的变量。

区别：

1) 作用域：

成员变量：针对整个类有效；局部变量：只在某个范围内有效；

2) 存储位置

成员变量：随着对象的创建而存在，随着对象的消失而消失，存储在堆内存中；

局部变量：在方法被调用，或者语句被执行的时候存在，存储在栈内存中；当方法调用完，或者语句结束后，就自动释放。

3) 生命周期

成员变量：随着对象的创建而存在，随着对象的消失而消失；

局部变量：当方法调用完，或者语句结束后，就自动释放；4)

初始值

成员变量：有默认初始值；

局部变量：没有默认初始值，使用前必须赋值。

java集合中谁有序，谁无序？

答：有序无序是指在进行插入操作时，插入位置的顺序性；先插的位置在前，后插的位置在后，则为有序，反之无序；

实现了List接口的集合类全部有序，如ArrayList、LinkedList；实现了set接口的集合类中，HashSet无序，TreeSet有序；实现了map接口的集合中，HashMap无序，TreeMap有序

java的反射机制是什么？有什么作用？

答：java的反射机制是指在程序的运行状态中，对于任意一个类，都能知道各类的所有属性和方法；对于任意一个对象，都能调用它的任意一个属性和方法。这种动态获取程序信息以及动态调用对象的功能，称为JAVA语言的反射机制。

作用：1）有助于观察和操纵应用程序的运行时行为；

2）有助于调试或测试程序，因为我们可以直接访问方法、构造函数和成员字段；

3）即使事前不知道某个方法，我们也可以通过名字调用该方法；

说说JAVA的类加载机制？

答：JAVA的类加载机制是指在JAVA程序运行时，将类文件加载到内存中的一系列步骤。主要分为三个过程：加载、连接和初始化，这三个过程的顺序是固定的。

JAVA的类加载机制遵循“按需加载”的原则，也就是说，只有在需要用到某个类的时候，才会将这个类的相关信息加载到内存中。这种按需加载的设计使得JAVA程序具备了很好的灵活性和效率。

undolog的作用？在MVCC中有没有用到？

答：undolog有两个作用：提供回滚和多个行版本控制(MVCC)。有用到。

介绍一下MVCC

答：MVCC（Multi-VersionConcurrencyControl）是数据库系统中一种并发控制机制，用于处理多个事务同时读取和修改数据库的情况。它能够确保并发事务之间的隔离性，提供高并发性和数据一致性。

在MVCC中，每个事务在修改数据库时会创建一个新的版本，通过版本号或时间戳来标识。每个版本对应一个事务的快照（Snapshot）或历史视图（HistoricalView），事务的读操作只能看到在它开始之前已经提交的版本，而不受其他并发事务的影

响。这种方式实现了并发读取的隔离性，避免了读取过程中的数据不一致性。

MVCC的实现通常需要额外的存储空间来管理版本信息，主要包括以下几个关键组件：

1.数据行版本链：每个数据行都会维护一个版本链表，其中存储了不同版本的数据内容和对应的事务ID或时间戳。

2.事务ID或时间戳：每个事务在开始时会被分配一个唯一的事务ID或时间戳，用于确定事务的读写顺序和版本的可见性。

3.快照读（SnapshotRead）：当一个事务进行读操作时，会基于自己的事务ID或时间戳，查找适合的可见版本进行读取，而不会受到其他并发事务的影响。

4.写操作和提交：当一个事务进行写操作时，会创建一个新的数据版本，并将事务ID或时间戳和数据更新写入版本链表。当事务提交时，新的版本才会对其他并发事务可见。

MVCC的好处在于可以提供高度的并发性能，避免了传统的锁机制带来的串行化问题。每个事务可以独立读取数据，即使是在其他并发事务修改同一数据的情况下，仍能保持一定的隔离性。该机制在一些流行的数据库系统中得到广泛应用，例如PostgreSQL和Oracle。然而，MVCC也会带来额外的存储和管理开销，并且需要较为复杂的实现逻辑来保证隔离性和一致性。

红黑树的特性？适用场景？

答：红黑树是一种特殊的二叉查找树。红黑树的每个节点上都有存储位表示节点的颜色，可以是红或者黑；

特性：

- 1) 每个节点或者是黑色，或者是红色；
- 2) 根节点必须是黑色；
- 3) 叶子结点都是黑色，且为null；
- 4) 连接红色节点的两个子节点都为黑色，也就是说红黑树不会出现相邻的红色节点；
- 5) 从任意节点出发，到其每个叶子节点的路径中包含相同数量的黑色节点
- 6) 新加入到红黑树的节点为红色节点；

适用场景：

- 1) 数据库索引
- 2) 文件系统
- 3) 线程调度
- 4) 并发数据实现
- 5) 缓存实现
- 6) IP路由表

为什么hashmap使用红黑树而不是平衡二叉树？

答：HashMap使用红黑树是因为红黑树相对于平衡二叉树有更好的性能表现。1) 首先，红黑树的平衡性能比平衡二叉树更好。红黑树的平衡性能是通过节点进行颜色标记和旋转操作来实现的，而平衡二叉树只能通过旋转操作来实现平衡。因此，红黑树的平衡性能更好，可以更快地进行插入、删除和查找操作。2) 其次，红黑树的空间利用率比平衡二叉树更高。红黑树的节点结构比平衡二叉树的节点结构更紧凑，因此在存储大量数据时，红黑树的空间利用率更高。3) 最后，红黑树的实现比平衡二叉树更简单。红黑树的实现比平衡二叉树的实现更简单，因为红黑树的平衡性能是通过颜色标记和旋转操作来实现的，而平衡二叉树需要更复杂的平衡算法来实现平衡。因此，HashMap使用红黑树来实现内部的数据结构，以提高性能和空间利用率。

完全二叉树：深度为k，有n个结点的二叉树，当且仅当其每一个结点都与深度为k的满二叉树中编号从1到n的结点一一对应时，称为完全二叉树。

满二叉树：如果一棵二叉树只有度为0的结点和度为2的结点，并且度为0的结点在同一层上，则这棵二叉树为满二叉树

ArrayList的底层？初始化过程和扩容时机

答：ArrayList实现了List接口，是顺序容器，即元素存放的数据与放进去的顺序相同，允许放入null元素，底层通过数组实现。除该类未实现同步外，其余跟Vector大致相同。每个ArrayList都有一个容量(capacity)，表示底层数组的实际大小，容器内存储元素的个数不能多于当前容量。当向容器中添加元素时，如果容量不足，容器会自动增大底层数组的大小。前面已经提过，Java泛型只是编译器提供的语法糖，所以这里的数组是一个Object数组，以便能够容纳任何类型的对象。

初始化过程：ArrayList的底层是一个动态数组，ArrayList首先会对传进来的初始化参数initialCapacity进行判断

如果参数等于0，则将数组初始化为一个空数组，

如果不等于0，将数组初始化为一个容量为10的数组。

扩容时机：当数组的大小大于初始容量的时候(比如初始为10，当添加第11个元素的时候)，就会进行扩容，新的容量为旧的容量的1.5倍。

什么是接口幂等性？

答：接口幂等性就是用户对于同一操作发起的一次请求或者多次请求的结果是一致的，不会因为多次点击而产生副作用。比如说支付场景，用户购买了商品支付扣款成功，但是返回结果的时候网络异常，此时钱已经扣了，用户再次点击按钮，此时会进行第二次扣款，返回结果成功，用户查询余额发现多扣钱了，流水记录

也变成了两条，这就没有保证接口幂等性。

在并发量特别高的情况下是使用synchronized还是使用reentrantLock?

答：在并发量特别高的情况下，一般推荐使用reentrantLock，原因如下：1) 更高的性能：在java1.6之前，synchronized的性能一般比reentrantlock差一些。虽然在java1.6及之后的版本中，synchronized进行了一些优化，如偏向锁、轻量级锁等，但是在高并发的情况下，reentrantlock的性能通常会优于synchronized；

2) 更大的灵活性：reentrantLock比synchronized有更多的功能。例如：reentrantlock可以实现公平锁和非公平锁，而synchronized只能实现非公平锁；reentrantlock提供了一个condition类，可以分组唤醒需要唤醒的线程，而synchronized要么随机唤醒一个线程，要么唤醒所有的线程；reentrantlock提供了trylock方法，可以尝试获取锁，如果获取不到立即返回，不会像synchronized那样阻塞；

3) 更好的可控制性：reentrantLock可以中断等待锁的线程，也可以获取等待锁的线程列表，这在调试并发问题时非常有用，而synchronized无法响应中断，。但是，虽然reentrantlock在功能上比synchronized更强大，但也更复杂，使用不当容易造成死锁。而synchronized由JVM直接支持，使用更简单，不容易出错。所以，在并发量不高，对性能要求不高的情况下，可以考虑使用synchronized。

介绍下reentrantlock以及原理

答：reentrantlock是一种可重入锁，也就是说同一个线程可以多次获取同一个锁而不会发生死锁；

线程的几种状态?

答：1) New新建状态
2) Runnable运行状态；
3) blocked阻塞状态
4) waiting等待状态
5) Time_waiting超时等待状态
6) Terminated终止状态

什么是序列化?

序列化就是将对象转换为二进制格式的过程。对象转为二进制后，可以方便进行存储或者在网络中传输。

要将对象转为二进制，首先就要用一种数据结构来描述对象，这样进行对象的网络传输的时候，发送方和接收方才能按照约定好的格式来序列化、反序列化

对象。不同的序列化算法就是用不同的数据结构来描述对象。

二叉树与平衡二叉树的区别？

答：1) 二叉查找树的根节点是不可变的，左右两边结点层级差没有限制；平衡二叉树左右两边结点层级相差不大于1，通过旋转实现根节点可变，达到自平衡。
2) 相对于二叉查找树，平衡二叉树的查询效率高，但由于增加和删除节点时，为了保证自平衡会做连续的旋转操作，平衡二叉树的额外开销比较大，耗时相对较长。

上下文切换的过程？

答：什么是上下文？

通常所说的上下文指CPU的上下文，也就是CPU运行任何任务前，必须依赖的环境，包括CPU寄存器和程序计数器。

上下文切换就是先把一个任务的CPU上下文，也就是程序计数器和寄存器，保存起来，然后加载新任务的上下文到这些寄存器和程序计数器，最后再跳转到程序计数器所指的新位置，运行新任务。

具体流程：

- (1) 保存当前进程的硬件上下文（PC/PSW/SP和通用寄存器等）；
- (2) 修改当前进程的PCB，比如将其状态由运行改为就绪或者阻塞，并将该进程PCB加入相关队列；
- (3) 调度另外一个进程（这涉及调度）
- (4) 修改被调度进程的PCB（进程控制块），将其状态改为运行（系统上下文）
- (5) 将“当前进程”的存储管理数据改为被调度进程的存储管理信息（如页表、TLB）（用户级上下文）
- (6) 恢复新进程的硬件上下文（即现场），让PC指向新进程代码

内核态和用户态

答：内核态：运行操作系统程序，操作硬件；

用户态：运行用户程序

用户态到内核态，唯一途径是通过中断、异常、陷入机制；内核态到用户态，通过设置程序状态字PSW实现；

处于用户态执行时，进程所能访问的内存空间和对象受到限制，其所处于占有的处理器是可被抢占的

处于内核态执行时，则能访问所有的内存空间和对象，且所占有的处理器是不允许被抢占的。

sleep和wait有什么区别？

答：两种方法都是用来使线程进入休眠状态的；

1) 语法使用不同

wait方法必须配合synchronized一起使用，不然在运行时抛出异常；而**sleep方法**可以单独使用，无需配合**synchronized**一起使用；

2) 所属类不同

wait方法属于Object类的方法，而**sleep属于Thread类的方法**；

3) 唤醒方式不同

sleep方法具有主动唤醒的功能，而**wait方法只能被动唤醒**；

4) 释放锁资源不同

wait方法会主动的释放锁，而**sleep方法不会**；

5) 线程进入状态不同

调用**sleep方法**线程会进入**TIMED_WAITING**有时限等待状态，而调用无参数的**wait方法**会进入**WAITING**无时限等待状态

讲讲java的线程池？

答：线程池可以看作是线程的集合。它的工作主要是控制运行的线程的数量，处理过程中将任务放入队列，然后在线程创建后启动这些任务，如果线程数量超过了最大数量，超出数量的线程排队等候，等其它线程执行完毕，再从队列中取出任务来执行。它的主要特点为：线程复用；控制最大并发数；管理线程。

线程池原理

答：了解线程池工作原理主要分为三个方面:线程池状态，线程池核心参数，以及线程池的基本工作流程。

线程池的几种状态？

（线程池是一种多线程处理形式，处理过程中将任务添加到队列，然后在创建线程后自动启动这些任务）

答：五种

1) **RUNNING**:线程一旦被创建就处于此状态，任务数为0，能够接受新的任务，对已排队的任务进行处理；

2) **SHUTDOWN**: 不能接受新任务，但能处理已经排队的任务；调用线程池的**shutdown()**方法，线程池有**RUNNING**转变为**SHUTDOWN**状态；

3) **STOP**: 不接受新任务，也不处理正在排队的任务，并且会中断正在处理的任务。调用线程池的**shutdownNow()**方法，线程池由**RUNNING**或**SHUTDOWN**状态转变为**STOP**状态。

4) **TIDYING**: 任务队列为空且执行中任务为空时，就会转成此态；

5) **TERMINATED**: 线程池彻底终止。线程池在**TIDYING**状态执行完**terminated()**方

法后，就会由TIDYING转变为TERMINATED状态。

线程池的参数

答：有7个参数

线程池默认工作流程

- 1、线程在有任务的时候会创建核心的线程数`corePoolSize`
- 2、当线程满了（有任务但是线程被使用完）不会立即扩容,而是放到阻塞队列中,当阻塞队列满了之后才会继续创建线程。
- 3、如果队列满了,线程数达到最大线程数则会执行拒绝策略。
- 4、当线程数大于核心线程数,超过`KeepAliveTime`(闲置时间),线程会被回收,最终会保持`corePoolSize`个线程。

线程池配置无界队列了之后，拒绝策略怎么搞，什么时候用到无界队列

答：线程池使用无界队列后，无界队列可以确保任务不会被丢弃，并且可以处理任意数量的任务；在对任务的执行顺序没有特殊要求的时候，可以用到无界队列；使用了无界队列，实际上不会触发拒绝策略，拒绝策略在有界队列中使用，当队列已满并且无法接受更多任务时，拒绝策略会被触发，常见的拒绝策略包括：

- 1) **abortpolicy**: 默认的拒绝策略，当队列已满且线程池中的线程数已达到最大线程数时，新提交的任务会抛出异常；
- 2) **DiscardPolicy**: 当队列已满时，新提交的任务会被丢弃，不会给任何提示或错误；
- 3) **DiscardOldestPolicy**: 当队列已满时，将队列中最早进入的队列丢弃，为新提交的任务腾出空间；

4) **CallerRunsPolicy**: 当队列已满时, 新提交的任务由提交任务的线程来执行, 即将任务回退到调用者线程。

为什么需要泛型?

答: 1) 适用于多种数据类型执行相同的代码, 通过泛型, 可以复用一个方法;
2) 泛型中的类型在使用时指定, 不需要强制类型转换;

从继承的角度看, 将构造函数声明为私有会有什么作用?

答: 将构造函数声明为私有, 可以确保类以外的地方都不能直接实例化这个类。在这种情况下, 要创建这个类的实例, 唯一的办法就是提供一个公共静态方法, 就像工厂模式那样。此外, 由于构造函数是私有的, 这个类也不能被继承。

HashMap的底层实现? 他为什么是线程不安全的? 想要线程安全有什么措施?

答: **HashMap**的底层实现是数组加链表(或红黑树)的组合。具体来说, 它使用一个存储**entry**对象的数组来存储键值对, 这个数组被分成若干个桶**bucket**, 每个桶可以存储多个**entry**对象。当发生哈希冲突时, 即多个键映射到同一个桶时, 相应的链表(或红黑树)会被用于存储冲突的键值对。

不安全的原因: 这是因为**hashmap**的实现在多线程并发环境下没有进行同步处理。当多个线程同时对**hashmap**进行修改时, 可能会导致数据的不一致。例如, 当两个线程同时执行**put**操作时, 可能会引发死锁或数据覆盖问题。

要使hashmap线程安全, 可以采取以下措施:

1) 使用**ConcurrentHashMap**; 2) 使用**Collections.synchronizedMap()**; 3) 使用**synchronized**关键字、**ReentrantLock**等锁机制来保证线程安全。

为什么concurrentHashMap是线程安全的?

答: **ConcurrentHashMap**是Java提供的线程安全的哈希表实现, 它相较于普通的**HashMap**具有更高的并发性和线程安全性, 原因如下:

1.分段锁: **ConcurrentHashMap**内部使用了一种分段锁(**Segment**), 将整个哈希表分成多个小的段, 每个段由一个独立的锁控制。不同的线程可以同时访问不同的段, 从而提供了更高的并发性。

2.读写分离: **ConcurrentHashMap**将读操作和写操作分离, 读取操作不需要加锁。在并发读取的情况下, 不会阻塞其他读线程, 提高了读取的并发性能。只有在写操作时, 才会对对应的段进行加锁, 保证写操作的原子性。

3.原子性操作: **ConcurrentHashMap**提供了一些原子性操作方法, 如**putIfAbsent()**、**remove()**等, 这些操作可以保证在多线程环境下的原子性。使用这些原子操作, 可以避免由于并发操作导致的数据不一致问题。

4.happens-before关系: **ConcurrentHashMap**内部通过使用**volatile**修饰符、内存

屏障等实现了happens-before关系，保证了在多线程环境下的可见性和有序性。5. 线程安全的迭代器：ConcurrentHashMap的迭代器是弱一致性的，通过在遍历时对每个段加锁，并确保迭代器在整个遍历期间不会抛出

ConcurrentModificationException等异常。

总的来说，ConcurrentHashMap通过分段锁、读写分离、原子性操作和happens-before关系等机制，保证了在多线程环境下的线程安全性。它提供了更高的并发性和性能，适用于需要频繁读写的并发场景。注意，虽然

ConcurrentHashMap是线程安全的，但并不意味着所有的操作都是原子性的，如果需要进行复合操作，仍需额外的同步措施。

ConcurrentHashMap和HashTable?

答：ConcurrentHashMap和HashTable都是java中提供的线程安全的哈希表实现，用于存储键值对。它们的设计目的都是在多线程环境下保证数据的一致性和线程安全性。

他们的区别如下：

1) 锁机制不同：ConcurrentHashMap使用了分段锁，不同的段可以由不同的线程同时访问，有效的提高了并发性能。而Hashtable使用一个全局锁，只能由一个线程进行访问，因此在高并发场景下性能较差；

2) Null值：ConcurrentHashMap允许null作为键或值，hashtable不允许null值，否则会抛出异常；

3) 扩容：concurrentHashMap在扩容时只需要锁住部分桶，并发处理效率高；而Hashtable在扩容时需要锁住整个HashMap，性能较低；

总得来说，ConcurrentHashMap相对于HashTable来说，在处理高并发环境中的性能更好。在新的JAVA版本中，推荐使用ConcurrentHashMap来代替HashTable。

说一下ConcurrentHashMap中并发安全的实现

ConcurrentHashMap在Java1.7和Java1.8中的实现方式有所不同，但它们的共同目标都是提供高效的并发更新操作。下面我将分别介绍这两个版本的实现方式。

(1) Java1.7：在Java1.7中，ConcurrentHashMap内部使用一个Segment数组来存储数据。每个Segment对象包含一个HashEntry数组，每个HashEntry对象就是一个键值对。Segment对象是可锁定的，每个Segment对象都可以看作是一个独立的HashMap。在更新数据时，只需要锁定相关Segment对象，而不需要锁定整个HashMap，这样就可以实现真正的并发更新。Segment的数量默认为16，这意味着ConcurrentHashMap最多支持16个线程的并发更新。(2) Java1.8：在Java1.8中，ConcurrentHashMap的实现方式进行了改进。它取消了Segment数组，直接使用Node数组来存储数据。每个Node对象就是一个键值对。在更新数据时，使用CAS操作和synchronized来保证并发安全。具体来说，如果更新操作发生在链表的头节点，那么使用CAS操作；如果发生在链表的其他位置，或者发生在红黑树节点，那么使用synchronized。这样，ConcurrentHashMap可以支持更多线程的并发新。

总的来说，ConcurrentHashMap通过分段锁（Java1.7）或CAS+synchronized（Java1.8）的方式，实现了高效的并发更新操作，从而保证了并发安全。

你说高并发下 ReentrantLock 性能比 synchronized 高，那为什么 ConcurrentHashMap 在 JDK1.7 中要用 ReentrantLock，而 ConcurrentHashMap 在 JDK1.8 要用 Synchronized？

这是一个很好的问题。首先，我们需要明确一点，虽然ReentrantLock在某些情况下的性能优于synchronized，但这并不意味着在所有情况下都是这样。实际上，synchronized在JDK1.6及以后的版本中已经进行了大量的优化，例如偏向锁、轻量级锁等，使得在竞争不激烈的情况下，synchronized的性能已经非常接近甚至超过ReentrantLock。

在JDK1.7的ConcurrentHashMap中，使用ReentrantLock（Segment）是为了实现分段锁的概念，即将数据分成多个段，每个段独立加锁，从而实现真正的并发更新。这种设计在当时是非常先进和高效的。

然而，在JDK1.8的ConcurrentHashMap中，为了进一步提高并发性能，引入了更复杂的数据结构（如红黑树）和更高效的并发控制机制（如CAS操作）。在这种情况下，使用synchronized比ReentrantLock更加简单和高效。首先，synchronized可以直接与JVM进行交互，无需用户手动释放锁，减少了出错的可能性。

其次，synchronized在JDK1.6及以后的版本中已经进行了大量的优化，性能已经非常接近ReentrantLock。最后，synchronized可以与其他JVM特性（如偏向锁、轻量级锁、锁消除、锁粗化等）更好地配合，进一步提高性能。

总的来说，选择使用ReentrantLock还是synchronized，需要根据具体的需求和使用场景来决定。在JDK1.8的ConcurrentHashMap中，使用synchronized是一种更加合理和高效的选择。

有哪些并发安全的实现方式

答：1) **synchronized**关键字：这是最基本的并发安全实现方式，它可以保证同一时刻最多只有一个线程执行该段代码，从而保证了类的实例状态的线程安全。

2) **volatile**关键字：volatile关键字可以保证变量的可见性和禁止指令重排序，但不能保证复合操作的原子性。它通常用于标记状态变量。

3) **Lock**接口和其实现类：例如ReentrantLock、ReentrantReadWriteLock等。相比于synchronized，Lock接口提供了更强大的功能，例如尝试获取锁、可中断的获取锁以及获取公平锁等。

4) 原子类：例如AtomicInteger、AtomicLong、AtomicReference等。这些类通过CAS（CompareandSwap）操作实现了原子性更新。

5) 并发集合：Java提供了一些线程安全的集合类，例如ConcurrentHashMap、CopyOnWriteArrayList等。

6) **ThreadLocal**类：ThreadLocal可以为每个线程创建一个单独的变量副本，每个线程都只能操作自己的副本，从而实现了线程隔离，保证了线程安全。

HashSet如何检查重复？

答：HashSet检查重复是通过哈希码和equals方法实现的。HashSet使用哈希算法将元素的值转换为哈希码并存储在哈希表中。当要添加一个新元素时，HashSet会先计算该元素的哈希码，并查找哈希表中是否已经存在相同的哈希码。如果存

在相同的哈希码，HashSet就会调用该元素的equals方法进行比较，如果equals方法返回true。HashSet就会认为该元素已经存在于集合中，不会将其添加到集合中。如果哈希码没有冲突，或者哈希码冲突但是equals方法返回false，HashSet就会将该元素添加到集合中。因此，HashSet检查重复是通过哈希码和equals方法实现的。

==和equals（）的区别？

答：在java中，==运算符和equals（）方法都可以用于比较两个对象。但是它们的比较方式是不同的：

==运算符比较的是两个对象的引用是否相等，即它们是否指向内存中的同一对象；equals()方法，在默认情况下，比较的是两个对象的内存地址是否相同，即比较引用是否相同。如果要判断对象的内容是否相同，则需要重写equals（）方法。

Threadlocal的原理是什么？

答：它是Thread的局部变量，是每个Thread里面都有一个ThreadLocalMap，key是ThreadLocal，value是我们存的Object，当我们调用set的时候，就会去当前线程所持有的ThreadLocalMap放入一个K-v对。k是当前的ThreadLocalMap对象，value是Object，当我们调用get的时候，会去ThreadLocalMap中，找到当前ThreadLocal对象对应的value返回。

（ThreadLocal是每个线程都持有一个吗？不是所有线程共享的吗？答：是的）

（线程里面的ThreadLocalMap是用static变量修饰的吗？答：是的）

new和malloc的区别？

答：1）new是操作符，malloc是函数；

2）new开辟内存失败抛出异常，malloc申请内存失败返回NULL；3）

new开辟的内存大小不需要指定，malloc需要指定。

4）new开辟的内存的类型会自动推导，不需要强转，malloc需要强转；5）实现原理：new会先调用operatornew函数，申请足够的内存。然后调用类型的构造函数，初始化成员变量，最后返回自定义类型指针。

malloc是库函数，只能动态的申请和释放内存，没有初始化功能，需要用户自己初始化。

Vector和list的区别？

答：1) **vector**的随机访问效率高，但在插入和删除时（不包括尾部）需要挪动数据，不易操作。
2) **List**的访问要遍历整个链表，它的随机访问效率低。但对数据的插入和删除操作等都比较方便，改变指针的指向即可。
3) **list**是单向的，**vector**是双向的。
4) **vector**中的迭代器在使用后就失效了，而**list**的迭代器在使用之后还可以继续使用。

ArrayList和vector区别？

答：1) **Vector**是线程安全的，**ArrayList**是线程不安全的；2) **ArrayList**在性能方面要优于**Vector**；
3) 当扩容时，**Vector**默认增长一倍，而**ArrayList**是增长50%。

创建对象的方法？

答：五种方法：

- 1) 字面量创建对象（每个对象都是独立的）用于创建单个对象；
- 2) 内置构造函数创建对象；
- 3) 工厂函数创建对象（可以根据需要传不同参数，创建多个对象）；
- 4) 自定义构造函数创建对象（通过**new**关键字简化步骤和代码，和工厂函数作用相同）；
- 5) **class**类创建对象（代码结构清晰）

如何保证java线程的安全性？

答：导致线程不安全的原因，主要有三个：

- 1) 原子性：一个或者多个操作在CPU执行过程中被中断；
 - 2) 可见性：一个线程共享变量的修改，导致另一个线程不能立即看到；
 - 3) 有序性：程序执行的顺序没有按照代码的先后顺序执行；
- 针对原子性：

1) **JDK**提供的**Atomic**类，通过**CAS**来保证原子性；2) **java**提供了各种锁机制，比如**synchronized**关键字；**针对可见性：**

- 1) 使用**synchronized**关键字加锁来解决；
- 2) **java**提供**volatile**关键字，比**synchronized**关键字性能更优；**针对有序性：**

- 1) 使用**synchronized**关键字定义同步代码块或同步方法；
- 2) 通过**lock**接口来保证有序性；

抽象工厂模式和工厂模式的区别？

答：普通工厂模式：生产具体的产品，创建的产品是类；

抽象工厂模式：生产抽象的产品，创建的产品是接口；

普通工厂模式在于只要创建当前的类，并不用实现里面的具体详情，所以用法也很明显，比较单一并不复杂的公用逻辑可以用此方法实现，而相对多样性的可以考虑抽象工厂模式

抽象工厂模式主要在于它定义了具体产品，也就是我们自己在写抽象工厂的时候，得实现多种产品方案，如果要修改必须修改工厂类。

堆排序的基本思想

答：1) 将待排序序列构造成一个大顶堆；

2) 此时整个序列的最大值就是顶堆的根节点；

3) 将其与末尾元素进行交换，此时末尾元素就是最大值；

4) 然后将剩余N-1个元素重新构成一个堆，这样会得到n个元素的第二大值，如此反复执行，便能得到一个有序的序列了；

什么是平衡二叉树？

答：它是一棵空树或它的左右两个子树的高度差的绝对值不超过1，并且左右两个子树都是一棵平衡二叉树。

说说你知道的序列化方式

（序列化：是将对象的状态信息转化为可以存储或传输的形式过程）

答：1) **JAVA原生序列化**。Java类通过serializable接口来实现该类对象的序列化。这个接口非常特殊,没有任何方法,只起标识作用。

2) **JSON序列化**。是一种轻量级的数据交换格式。JSON序列化就是将数据对象转换为JSON字符串。在序列化过程中抛弃了类型信息,所以反序列化时只有提供类型信息才能准确地反序列化。

反射的作用？

答：1) 获取已装载类的成员变量信息；2) 获取已装载类的方法；3) 获取已装载类的构造方法信息。

反射是否破坏了控制范围这个目的？

答：是的，JAVA反射机制有时可以被用来破坏访问控制范围这个目的。在java中，类的成员可以设置为私有、受保护或默认访问修饰符，以限制对其的访问。

然而，使用反射机制可以绕过这些访问修饰符的限制，访问和修改本来不应该被

访问的类成员。通过反射，可以获取并调用私有方法、访问私有字段，甚至创建对象时忽略构造函数的访问限制。

java设计模式有哪些？

答：1) 观察者模式

又称为发布-订阅模式，定义了对象之间一对多依赖关系，当目标对象的状态发生改变时，它的所有依赖者都会收到通知；

2) 抽象工厂模式

主要用于创建相关的家族。

3) 单例设计模式

可以确保系统中某个类只有一个实例，该类自行实例化并向整个系统提供这个实例

例的公共访问点，除了该公共访问点，不能通过其他路径访问该实例。

优点：

系统中只存在一个共用的实例对象，无需频繁创建和销毁对象，节约了系统资源，提高系统的性能；

可以严格控制客户怎么样以及何时访问单例对象；

单例模式的写法有好几种，主要有三种：懒汉式单例、饿汉式单例、登记式单例

单例模式中懒汉模式和饿汉模式的区别

答：1) 饿汉就是类一旦加载，就完成单例初始化，保证`getInstance`的时候单例是已经存在的；而懒汉比较懒，只有当调用`getInstance`的时候，才会去初始化这个单例。

2) 饿汉模式天生就是线程安全的，可以直接用于多线程而不会出现问题；懒汉模式本身就是非线程安全的。

synchronized和volatile的区别？

答：1) `synchronized`作用于代码或者方法；`volatile`是变量修饰符；

2) `synchronized`是一种锁机制，存在阻塞问题和性能问题；而`volatile`并不是锁，所以不存在阻塞和性能问题；

3) `synchronized`保证三大特性，`volatile`不保证原子性；

4) `synchronized`标记的变量可以被编译器；`volatile`标记的变量不会被编译器优化

语法糖

所谓的语法糖，其实就是指java编译器把*.Java源码编译为*.class字节码的过程中，自动生成和转换的一些代码，主要是为了减轻程序员的负担，算是java编译器给我们的一个额外的福利。

静态变量和实例变量的区别？

答：静态变量也称为类变量，归全类所有，它不依赖于某个对象，可通过类名直接访问；而实例变量必须依存与某一实例，只能通过对象才能访问它；

什么是集合？

答：集合就是一个放数据的容器，准确的说是放数据对象引用的容器；
集合类存放的都是对象的引用，而不是对象本身；
集合类型主要有3种：set、list、map

集合与数组的区别是什么？

答：1) 数组是固定长度的，集合是可变长度的；
2) 数组是可以存储基本数据类型，也可以存储引用数据类型；集合只能存储引用数据类型；
3) 和数组对比，对象的大小不确定，因为集合是可变长度的，数组需要提前定义大小；

说说集合的框架（或：常用的集合类有哪些？）

答：map接口和collection接口是所有集合框架的父接口；
collection接口的子接口包括：set接口和list接口；
其中，set接口的实现类主要有：HashSet、TreeSet等；List接口的实现类主要有：ArrayList、LinkedList、Vector等；Map接口的实现类主要有：HashMap、TreeMap等

Hashmap如何去重？

答：当我们向一个HashMap中插入元素时，HashMap会根据这个key的equals和hashCode方法进行判断，如果两个key的hashCode值相同，且equals返回True，那么HashMap将认为这是同一个key，后续插入相同key的键值对会将旧值替换为新值。

Mysql数据库

非关系型和关系型数据库的区别？

答：关系型数据库：

使用表格形式来存储数据，并通过主键和外键的关系将表格连接在一起；数据的结构是固定的，每个字段必须有特定的数据类型。

强制执行ACID(Atomicity,Consistency,Isolation,Durability)事务。适用于处理复杂的关系数据。

例如：MySQL,PostgreSQL,SQLServer等。

非关系型数据库：

不使用表格的形式来存储数据，而是使用键值对、文档或图形等不同的数据结构。数据结构可以动态地更改，字段可以没有特定的数据类型。

没有ACID事务的强制执行。

适用于处理高速读写、大量数据和弹性扩展等需求。

例如：MongoDB,Cassandra,CouchDB等。

mysql与clickhouse

答：clickhouse也是一种关系型数据库，最大的区别就是clickhouse是列式存储。列存储方式具备一种天然的优势，就是做统计分析，聚类分析，适合于联机分析处理场景，适用于海量数据处理、复杂查询、实时数据分析。它不支持事务，不适合处理数据的频繁修改以及删除操作，对于删除和修改会消耗大量的性能，特别是单条数据修改，尽量批量写入。

mysql中的高效查询语句有哪些？

答：在MySQL中，以下是一些常见的高效查询语句：

- 1) 使用索引：确保在经常查询的列上使用索引。索引可以加快查询速度，并且可以避免全表扫描。使用`CREATE INDEX`语句来创建索引，使用`EXPLAIN`语句来检查查询是否使用了索引。
- 2) 优化WHERE子句：在WHERE子句中使用合适的条件来减少结果集的大小，这样可以提高查询的效率。避免在WHERE子句中使用不必要的函数或表达式，因为这会使索引无效。
- 3) 避免使用SELECT*：只选择你需要的列，而不是选择所有列。这样可以减少网络传输的数据量和查询的执行时间。
- 4) 使用LIMIT来限制结果集大小：如果只需要查询结果中的一部分数据，可以使用LIMIT来限制返回的行数。这样可以避免检索和传输不必要的数据。
- 5) 避免使用子查询：子查询往往会导致性能下降。如果可能的话，可以使用连接（JOIN）来替代子查询。
- 6) 缓存复杂查询结果：如果某个查询的结果不经常变化，并且执行时间较长，可以将结果缓存起来，并在需要的时候直接使用缓存的结果。
- 7) 使用合适的数据类型：使用合适的数据类型可以减小存储空间的占用，并提升查询的执行效率。
- 8) 定期进行表优化：使用`OPTIMIZE TABLE`语句来定期优化表，以减少碎片和提升性能。

索引失效的情况

答：1) like查询以“%”开头；
2) or语句前后没有同时使用索引；
3) 组合索引中不是使用第一列索引；
4) 在索引列上使用“isnull”或“isnotnull”操作；5)
在索引字段上使用“not”，“!=”等。

InnoDB索引与MyISAM索引实现的区别是什么？

答：InnoDB和MyISAM都使用B+树来实现索引：
1) MyISAM索引文件和数据是分开存储的，索引文件仅保存数据记录的地址；
InnoDB的索引文件和数据是存储在一起的，它的数据文件本身就是索引文件。
2) InnoDB要求表必须有主键，MyISAM可以没有；

说说分区索引

答：分区索引就是简单的把一个索引分成多个片段。通过把一个索引分成多个片段，可以访问更小的片段，速度也更快，并且可以把这些片段分别存放在不同的磁盘驱动器上。

如果不加事务一个方法中有两条sql，第二条出现了问题，但是第一条正常执行，一般怎么办？

答：如果不加事务，一个方法中有两条SQL，第二条出现了问题，但是第一条正常执行，原因是因为SQL操作是自动提交的，当第一条SQL执行完成后，数据库就会自动提交，这就导致了第一条SQL的更改已经提交到数据库中，而第二条SQL因为出现了问题，没有被执行。

如果加了事务，第二条出现错误，第一条会回滚，及状态恢复到调用之前的状态

MyISAM和InnoDB分别是什么索引？聚集还是非聚集？

答：1) **MyISAM(非聚集)**

使用B+树作为索引结构，索引文件和数据分开存储，叶节点的data域存放的是数据记录的地址。

2) **InnoDB（聚集索引）**

索引文件和数据是存储在一起的，它的数据文件本身就是索引文件。

InnoDB必须要有主键，如果我们定义了主键（PrimaryKey），那么InnoDB会选择其作为聚集索引；如果没有显式定义主键，则InnoDB会选择第一个不包含有NULL值的唯一索引作为主键索引；

InnoDB为什么要用自增id作为主键？

答：如果表使用自增主键，那么每次插入新的记录，记录就会顺序添加到当前索引节点的后续位置，当一页写满，就会自动开辟一个新的页。

如果使用非自增主键（比如身份证号或者学号等），由于每次插入主键的值近似于随机，因此每次新纪录都要被插到现有索引页的中间某个位置，频繁的移动、分页操作造成了大量的碎片，得到了不够紧凑的索引结构，后续不得不通过`optimizetable`来重建表并优化填充页面。

说说分库与分表设计

答：分库与分表的目的在于，减小数据库的单库单表负担，提高查询性能，缩短查询时间。

通过分表，可以减少数据库的单表负担，将压力分散到不同的表上，同时因为不同的表上的数据量少了，起到提高查询性能，缩短查询时间的作用，此外，可以很大的缓解表锁的问题。

聚集索引与非聚集索引的区别

答：1）聚集索引是将索引结构的和数据放到一起的索引，索引的叶子结点存放数据；非聚集索引是将索引结构和数据分开存储的索引，索引的叶子结点指向了数据对应的位置；
2）一个表只能有一个聚集索引，但是可以有多个非聚集索引；
3）聚集索引插入数据时速度比非聚集索引慢，但是查询数据比非聚集索引速度快。

如果查非聚簇索引，能不能不回表，什么情况不回表？

答：可以不回表，当查询语句所要求的字段全部命中的时候，可以不进行回表查询。一个索引包含所有需要查询字段的值，称为“覆盖索引”。

举个简单的例子，假设我们在员工表的年龄上建立了索引，那么当进行`select score from student where score > 90`的查询时，在索引的叶子结点上，已经包含了`score`信息，不会再进行回表查询。

事务四大特性是什么？实现原理又是什么？

答：ACID：原子性、一致性、隔离性、持久性。

原子性：一个事务中的所有操作，要么全部完成，要么全部失败回滚；

一致性：在事务开始之前和事务结束以后，数据保持一致。

隔离性：一个事务执行的过程中，其他事务不会对其他产生影响。

持久性：事务处理结束后，对数据的修改就是永久的，即便系统故障也不会丢失。

实现原理：

- 1) 原子性和持久性：**redolog**来保证，**redolog**又叫做重做日志，是物理日志，事务提交的时候，必须先将事务的所有日志写入**redolog**持久化，到事务的提交操作才算完成；
- 2) 一致性：**undolog**来保证，**undolog**是逻辑日志，记录了事务的insert、update、delete操作，回滚的时候做相反的delete、update、insert操作来恢复数据；
- 3) 隔离性：由数据库锁以及MVCC机制来实现的。

事务的并发会产生什么问题？事务隔离级别，每个级别会解决什么问题，MySQL

默认是哪个级别？

答：事物的并发问题：

当前事务中读到其他事务未提交的数据。

不可重复读：在事务A中先后两次读取同一个数据，但是两次读取的结果不一样。

幻读：在事务A中按照某个条件先后两次查询数据库，两次查询结果的行数不同。

有四个隔离级别，从低到高分别是：读未提交、读已提交、可重复读、串行化；

读未提交：可以读到其他事务未提交的数据，不能解决以上任何问题；

读已提交：只能读到其他事务已经提交的数据，可以解决脏读的问题；

可重复读：先后读取同一个数据，结果是一样的；可以解决脏读和不可重复读问题；

串行化：按照顺序挨个执行。可以解决脏读、不可重复读、幻读所有问题；

MySQL的默认隔离级别就是Repeatable-read，可重复读。

MySQL的隔离级别是如何实现的？

答：MySQL的隔离级别是通过MVCC和锁机制来实现的；

1) 读未提交隔离级别最低，没有加锁；

2) 读已提交和可重复读隔离级别通过MVCC来实现；

3) 串行化是通过锁机制实现的，读加共享锁，写加排他锁，读写互斥。如果有未提交的事务正在修改某些行，所有select这些行的语句都会阻塞；

MySQL如何优化？

答：1) 使用索引：创建合适的索引可以大大加快查询速度；

2) 使用缓存：对于经常访问的查询结果，可以考虑使用缓存技术

3) 优化查询：尽可能使用简单的查询语句

4) 优化表结构：避免使用大型、不必要的列和表，以及尽可能使用小型数据类型；

5) 避免使用select*：只选择需要的列可以减少查询时间和减轻数据库负载。

6) 避免使用子查询：子查询可能会增加查询时间

不可重复读与幻读的区别是什么？

答：1) 前者是数据变了，后者是数据的行数变了。
2) 不可重复读侧重于修改，幻读侧重于新增或者删除；
3) 解决不可重复读的问题只需锁住满足条件的行，解决幻读需要锁表

脏读和不可重复读的区别在于

答：脏读读到的是其他事务未提交的数据，不可重复读读到的是其他事务已经提交的数据

讲一下Mysql主从复制？

答：主从复制解决的问题就是在我们平时开发的程序中操作数据库的时候，大多数的情况查询的操作大大超过了写的操作，也就是说对数据库读取压力比较大的问题，为了保证数据库的效率，我们一般把读的操作放在从库执行，比较耗时与频繁I/O的操作放在主库执行。这样做的意义就是减轻主库操作的压力，主从库可以各行其事，提高了数据库的操作效率。

mysql主从复制如何保证数据的一致性？

答：1) 在主服务器上开启二进制日志binarylog:主服务器上所有的数据修改操作都记录在二进制日志中，包括插入、更新和删除等操作。二进制日志中的操作会按照顺序进行记录，并生成一个唯一的日志位置标识符；
2) 在从服务器上启动复制进程：从服务器连接到主服务器，并开始复制主服务器的二进制日志。从服务器会从主服务器获取二进制日志的内容，并将其写入到自己的重做日志redolog中。
3) 从服务器应用重做日志：从服务器会将重做日志中的内容应用到自己的数据库中，实现数据域主服务器的同步；
4) 保证复制的事务性：为了保证数据的一致性，通常在主服务器执行的操作都是事务性的。从服务器在接收到主服务器的事务时，会将其作为一个整体执行，保证了数据的一致性。
5) 可以通过监控复制过程的状态和延迟来确保数据的一致性。

MySQL常见的存储引擎InnoDB、MyISAM的区别？

答：1) 事务：MyISAM不支持，InnoDB支持；
2) 锁级别：MyISAM表级锁，InnoDB行级锁及外键约束；
3) MyISAM存储表的总行数；InnoDB不存储总行数；
4) MyISAM采用非聚集索引，B+树叶子结点存储指向数据文件的指针。InnoDB采用聚集索引，B+树叶子结点存储数据。
5) 适用场景：MyISAM适合插入不频繁，查询非常频繁的场景；如果执行大量

的SELECT，MyISAM是更好的选择；
InnoDB适合可靠性要求比较高，或者要求事务，表更新和查询都相当的频繁的场景，比如大量的INSERT或者UPDATE操作；

MySQL为什么使用B+树，而不是使用其他？B+树的特点？

答：B+树索引是B-Tree的改进版本。数据都在叶子结点上，并且增加了顺序访问指针，每个叶子节点都是指向相邻的叶子结点的地址。B+树的遍历只需要遍历叶子节点就可以实现整棵树的遍历，而B-Tree需要获取所有节点，相比之下B+树的效率更高。

Hash:虽然可以快速定位，但是没有顺序，IO复杂度高；

二叉树：树的高度不均匀，不能自平衡，查找效率跟数据有关（树的高度），并

且IO代价高；

红黑树：树的高度随着数据量增加而增加，IO代价高。

为什么说B+树比B树更适合实际应用中操作系统的文件索引和数据库索引？

答：B+树的磁盘读写代价更低，且其查询效率更加稳定，而且B+树只要遍历叶子节点就可以实现整棵树的遍历，而B树只能中序遍历所有节点，效率太低。

B+树的特点是什么？

- 1) 所有关键字都出现在叶子节点的链表中，且链表中的关键字恰好是有序的；2) 不可能在非叶子节点命中；
- 3) 非叶子节点相当于是叶子节点的索引，叶子节点相当于是存储数据的数据层；

建立索引的使用场景

答：在最频繁使用的、用以缩小查询范围的字段，需要排列的字段上建立索引。

不宜：

- 1) 对于查询中很少涉及列或者重复值比较多的列；
- 2) 对于一些特殊的数据类型，不宜建立索引，比如文本字段（text）等。

drop、delete与truncate的区别

答：drop、delete与truncate都表示删除，但是三者有一些差别：

- 1) Delete用来删除表的全部或者一部分数据行，可回滚。会触发这个表上的触

发器；

2)Truncate删除表中的所有数据，这个操作不能回滚，也不会触发这个表上的触发器，Truncate比delete更快，占用的空间更小；

3)Drop命令从数据库中删除表，所有的数据行、索引和权限也会被删除，所有的DML触发器也不会被触发，这个命令也不能回滚。

因此，在不再需要一张表的时候，用drop;在想删除部分数据行的时候，用delete;在保留表而删除所有数据的时候用truncate。

Mysql中delete和drop的区别

答：1) delete是一个数据操作语言命令，而drop是一个数据定义语言命令；2) delete命令用于从表中删除部分或全部元组。而drop命令用于从数据库中删除一个或多个表，可以删除表的所有相关数据、索引、约束等；

3) delete可以与where子句一起使用，但drop不能与任何命令一起使用；4) 可以使用delete命令执行回滚操作，因为它使用缓冲区；但在使用drop命令的情况下无法操作回滚，因为它直接处理实际数据；

5) 由于delete命令不删除表，因此不会释放任何空间；而drop命令会删除整个表，从而释放内存空间。

什么叫视图？游标是什么？

答：视图是一种虚拟的表，通常是有一个表或者多个表的行或者列的子集，具有和物理表相同的功能；

游标是将查询出来的结果集作为一个单元来有效的处理。一般不使用游标，但是需要逐条处理数据的时候，游标显得十分重要。

什么是存储过程？用来调用什么？

答：存储过程是一个预编译的SQL语句，使用存储过程比单纯SQL语句执行更快。调用：

1) 可以用一个命令对象来调用存储过程；

2) 可以供外部程序调用，比如:java程序。

内连接、自连接、外连接（左、右、全）、交叉连接的区别？

答：内连接：只有两个元素表相匹配的才能在结果集中显示；

外连接：

左外连接：左边为驱动表，驱动表的数据全部显示，匹配表的不匹配的不会显示；
右外连接：右边为驱动表，驱动表的数据全部显示，匹配表的不匹配的不会显示；
全外连接：连接的表中不匹配的数据全部会显示出来；
交叉连接：笛卡尔效应，显示的结果是连接表数的乘积。

MVCC讲一下

多版本并发控制，是MySQL的InnoDB存储引擎实现隔离级别的一种具体方式，用于实现提交读和可重复读这两种隔离级别。

MVCC的实现主要是为了提高数据库并发性能，用更好的方式去处理读-写冲突，做到即使有读写冲突时，也能做到不加锁，非阻塞并发读。

MVCC的特点就是在同一时刻，不同事物可以读取到不同版本的数据，从而可以解决脏读和不可重复读的问题。

MVCC实际上就是通过数据的隐藏列和回滚日志（`undolog`），实现多个版本数据的共存。这样的好处是，使用MVCC进行读数据的时候，不用加锁，从而避免了同时读写的冲突。

（通俗的来讲，数据库中同时存在多个版本的数据，并不是整个数据库的多个版本，而是某一条记录的多个版本同时存在，在某个事务对其进行操作的时候，需要查看这一条记录的隐藏列事务版本id，对比事务id并根据事务隔离级别去判断哪个版本的数据）。

什么是快照读和当前读？

答：快照读：读取的是记录数据的可见版本（有旧的版本）。不加锁，普通的select语句都是快照读；

当前读：读取的是记录数据的最新版本，显式加锁的都是当前读

limit的使用方法

答：limit start,size的含义：start是表示起始数据的行数、size表示展示的数据数量；特别需要注意的是MySQL中的第一条数据是从下标index=0开始算的。如果只给一个参数那么这个参数一定是size，start默认就是0

count(*)、count(1)、count(col)区别？

答：count(1)和count(*)都是统计表的总行数，两者执行结果相同；count(col)则是统计col列中不为空的总行数。如果该列存在索引，那么自动走索引，否则走全表扫描。

Mysql中索引都有哪些分类？

答:按数据结构分类: B+tree索引、Hash索引、Full-text索引;

按物理存储分类: 聚簇索引、二级索引;

按字段特性分类: 主键索引、普通索引、前缀索引;

按字段个数分类: 单列索引、联合索引

Mysql和Oracle什么区别？

答: mysql和oracle都是关系型数据库, 应用于各种平台。

1) mysql属于轻量型数据库; oracle属于重量型数据库;

2) mysql默认端口: 3306, 默认用户: root; oracle默认端口: 1521, 默认用户: system;

3) mysql的安装卸载很简单, oracle相对来说麻烦一点; 安装所占用的内存空间mysql比oracle多得多; 使用的时候oracle占用特别更大的内存空间;

4) mysql的管理工具较少, 在Linux下的管理工具的安装有时需要额外的包, 有一定的复杂性; oracle有很多成熟命令行、图形界面、web管理工具, 还有很多第三方的管理工具, 管理及其方便高效。

5) mysql默认用户是root, 用户下可以创建好多数据库, 每个数据库下还有好多表, 一般情况下都是使用默认用户, 不会创建多个用户; oracle创建一个数据库, 数据库下有好多用户, 不同用户下有很多表, 一般情况下只能创建一个数据库用。

mysql主从复制

答: 是指把数据从一个mysql主节点复制到一个或多个mysql从节点, 这就是mysql主从复制。

主从复制会把主节点中的所有数据库实例、特定数据库实例或特定表, 全部复制到从节点。

包含: 异步复制、半同步、以及全同步复制三种复制模式;

默认的复制是异步的, 主库在执行完客户端提交的事务后会立即将结果返给客户端;

半同步复制介于异步复制和全同步复制之间, 主库在执行完客户端提交的事务后不是立即返回给客户端, 而是等待至少一个从库接收到并写到relaylog中才返回给客户端;

全同步复制是指主库执行完一个事务后, 然后所有的从库都复制了该事务并成功执行完才返回成功信息给客户端;

物理日志和逻辑日志的区别？

答: 逻辑日志: 可以简单理解为记录的就是sql语句;

物理日志: 因为mysql数据最终是保存在数据页中的, 物理日志记录的就是数据页的变更;

mysql三大日志——binlog、redolog、undolog

答：redolog:是InnoDB存储引擎层的日志，物理日志。又称重做日志文件，用于记录事务操作的变化，记录的是数据修改后的值，不管事务是否提交都会记录下来。

binlog: 是MySQLServer层面的，又称为归档日志，属于逻辑日志，以二进制的形式记录的，用于记录数据库执行的写入性操作（不包括查询）信息。undolog: 主要记录了数据的逻辑变化，保存了事务发生之前的数据的一个版本，这样在发生错误时，就能回滚到事务之前的数据状态。

undolog的用途？

答：1) 事务回滚时，保证原子性和一致性；
2) 用于MVCC快照读

redolog和binlog的区别？

答：1) redolog是属于InnoDB层面，binlog是属于MySQLserver层面；2) redolog是物理日志，记录该数据页更新的内容；binlog是逻辑日志，记录的是这个更新语句的原始逻辑；
3) redolog是循环写，日志空间大小固定；binlog是追加写，是指一份写到一定大小的时候会更换下一个文件，不会覆盖；
4) redolog保证了事务的持久性，Binlog保证了事务的原子性和一致性。

存储引擎MyISAM和InnoDB的区别？

答：MyISAM: 不支持事务和外键约束，占用空间较小，访问速度快，表级锁定，支持全文索引，适用于不需要事务处理、单独写入或查询的应用场景；InnoDB: 支持事务处理、外键约束、占用空间比MyISAM大，支持行级锁定，5.5版本后支持全文索引，适用于需要事务处理、读写频繁，一致性要求高的应用场景。

mysql什么情况下索引会失效？（如果设置了合理的索引查询还是很慢可能是什么原因）

答：1) 字段类型不匹配；
2) Or语句前后没有同时使用索引；
3) like查询是以%开头；
4) 联合索引没有使用第一列索引；
5) 使用in, not in可能不会走索引；
6) 被索引字段使用了表达式计算；

7)被索引字段使用了内置函数;

unionall和union的区间?

答: unionall只是合并查询结果,并不会进行去重和排序操作,也就是全部的结果都展示; union会去除重复行,并进行默认规则的排序。

mysql中having和where的区别?

答: 1) 执行时机不同: where是分组之前进行过滤,不满足where条件,不参与分组; 而having是分组之后对结果进行过滤。

2) 判断时机不同: where不能对聚合函数进行判断,而having可以。

Mysql如何优化?

答: 1) 使用索引: 创建合适的索引可以大大加快查询速度;

2) 优化查询: 尽可能使用简单的查询语句

3) 优化表结构: 避免使用大型、不必要的列和表,以及尽可能使用小型数据类型;

4) 避免使用select*: 只选择需要的列可以减少查询时间和减轻数据库负载。

5) 避免使用子查询: 子查询可能会增加查询时间

6) 使用缓存: 对于经常访问的查询结果,可以考虑使用缓存技术

关系型数据库和非关系型数据库的区别?

答: 它们在数据组织和存储方式上有很大的差异。

1) 关系型数据库使用表格形式来存储数据,并通过主键和外键的关系将表格连接在一起,数据的结构是固定的,每个字段必须有特定的数据类型;强制执行ACID事务,适用于处理复杂的关系数据;

2) 非关系型数据库不适用表格的形式来存储数据,而是通过使用键值对、文档和图形等不同的数据结构。数据结构可以动态的更改,字段可以没有特定的数据类型。

主键索引与唯一性索引的区别

答: 1) 主键约束比唯一性索引的约束严格,在没有设定主键时,非空唯一性索引自动成为主键;

2) 主键索引所对应的属性列的值不能为空,唯一性索引对应的属性列的值可以为空;

3) 主键索引只有一个,唯一性索引可以有多个

数据库常见的四种索引

答：1) 普通索引：主要以B+树和哈希索引为主，任务是加快对数据的访问速度，常用于查询和排序的条件，值可以为空并且没有唯一性的限制；

2) 唯一性索引：与普通索引类似，不同的是唯一性索引，索引列的值必须是唯一的，但可以为空；

3) 主键索引：主键索引是一种特殊的唯一性索引，是创建在主键上的索引，所有属性列唯一且不能为空；

4) 全文索引：是利用查询关键字和查询列内容之间的相关度进行索引，可以利用全文索引来提高匹配的速度；

mysql有哪些情况不适合建立索引

答：1) 在where、groupby、orderby中用不到的字段；2) 数据量小的表；

3) 有大量重复数据的列

4) 很少查询的列

Mysql的执行引擎有哪些？有什么区别？适用于什么场景？

答：1) MyISAM

它是mysql默认的存储引擎，它是一种基于表的存储引擎，支持全文索引和压缩表等功能。MyISAM的优点是性能较好，适合读密集型的应用，但是不支持事务和行级锁定，所以在写密集型的应用中表现较差。

2) InnoDB引擎

InnoDB引擎是MySQL支持事务的标准引擎，支持外键、MVCC机制，具备较高的并发性、可靠性及安全性，是当今处理复杂web应用的一种最佳选择。InnoDB引擎的缺点除了速度慢外还有存储空间浪费等。

3) Memory引擎

Memory引擎适合处理非常小的和活动度较高的表，因为它会在内存中存储表的所有数据，不仅速度快，还可以降低磁盘I/O，但是Memory引擎的缺点是由于数据储存在内存中，当数据量很大时很快会占满内存，而且在mysql重启后数据会被清空，所以不适合储存大量数据。

Mybatis

MyBatis是什么？

答：Mybatis是一个半ORM（对象关系映射）框架，它内部封装了JDBC，开发时只需要关注sql语句本身，不需要花费精力去处理加载驱动、创建链接、创建statement等繁杂的过程。程序员直接编写原生态sql，可以严格控制sql执行能力，灵活度高。

Mybatis可以使用XML或注解来配置和映射原生信息，将POJO映射成数据库中的记录，避免了几乎所有的JDBC代码和手动设置参数以及获取结果集。

Mybatis优缺点

答：优点：与传统数据库访问技术相比，有以下优点：

- 1）基于SQL语句编程，相当灵活，不会对应用程序或者现有设计造成任何影响，SQL写在XML里，解除SQL与程序代码的耦合，便于统一管理；提供XML标签，支持编写动态SQL语句，并可重用；
- 2）与JDBC相比，减少了50%以上的代码量，消除了JDBC大量冗余的代码，不需要手动开关连接；
- 3）很好的与各种数据库兼容（因为Mybatis使用jdbc来连接数据库，所以只要JDBC支持的数据库，mybatis都支持）
- 4）提供映射标签，支持对象与数据库的ORM字段关系映射；提供对象关系映射标签，支持对象关系组件维护；
- 5）能够与Spring很好的集成

缺点：

- 1）SQL语句的编写工作量较大，尤其当字段多、关联表多时，对开发人员编写SQL语句的功底有一定的要求。
- 2）SQL语句依赖于数据库，导致数据库移植性差，不能随意更换数据库。

ORM是什么？

答：ORM，对象关系映射，是为了解决关系型数据库数据与简单Java对象（POJO）的映射关系的技术。简单的说，ORM是通过使用描述对象和数据库之间映射的元数据，将程序中的对象自动持久化到关系型数据库中。

JDBC编程有哪些不足之处，Mybatis是如何解决的？

答：1) 数据库连接创建、释放频繁造成系统资源浪费，从而影响系统的性能，如果使用数据库连接池可以解决此问题；

解决：在mybatis-config.xml中配置数据连接池，使用连接池管理数据库连接；2) SQL语句写在代码中造成代码不易维护，实际应用sql变化的可能较大，sql变动需要改变java代码；

解决：将sql语句配置在XXXXmapper.xml文件中与java代码分离；

3) 向sql语句传参数麻烦，因为sql语句的where条件不一定，可能多也可能少，占位符需要和参数一一对应；

解决：mybatis自动将java对象映射至sql语句。

4) 对结果解析麻烦，sql变化导致解析代码变化，且解析前需要遍历，如果能将数据库记录封装成pojo对象解析比较方便；

解决:mybatis自动将sql执行结果映射至java对象。

MyBatis编程步骤是什么样的？

答：1.创建SqlSessionFactory

2.通过SqlSessionFactory创建SqlSession

2.通过SqlSession执行数据库操作

3.调用session.commit()提交事务；

4.调用session.close()关闭会话；

Mybatis的功能架构是怎样的？

答：分为三层：

1) API接口层：提供给外部使用的接口API，开发人员通过这些本地API来操纵数据库。接口层一接收到调用请求就会调用数据处理层来完成具体的数据处理。

2) 数据处理层：负责具体的SQL查找、SQL解析、SQL执行和执行结果映射处理等。它的主要目的是根据调用的请求完成一次数据库操作。

3) 基础支撑层：负责最基础的功能支撑，包括连接管理、事务管理、配置加载和缓存处理，这些都是共用的东西，将他们抽取出来作为最基础的组件，为上层的数据处理层提供最基础的支撑。

#{}和\${}的区别?

答：#{}是占位符，预编译处理；\${}是拼接符，字符串替换，没有预编译处理；

- 1) Mybatis在处理#{}时，#{}传入参数是以字符串传入，会将SQL中的#{}替换为？号，调用PreparedStatement的set方法来赋值；
- 2) #{}可以有效的防止SQL注入，提高系统安全性；\${}不能防止sql注入；
- 3) #{}变量替换是在DBMS中；\${}的变量替换是在DBMS外；

计算机网络

简述tcp和udp的区别?

- 答：1) TCP是传输控制协议，UDP是用户数据表协议；
2) TCP长连接，UDP无连接；
3) TCP可靠，保证数据正确性、顺序性；UDP不可靠，可能丢数据；4)
TCP适用于少量数据，UDP适用于大量数据传输；
5) TCP速度慢，UDP速度快。

基于TCP的协议：http、FTP、SSH、SMTP

基于UDP的协议：DNS、TFTP、SNMP

为什么要三次握手?

答：如果只有一次握手，Client不能确定与Server的单向连接，更加不能确定Server与Client的单向连接；

如果只有两次握手，Client确定与Server的单向连接，但是Server不能确定与Client的单向连接；

只有三次握手，Client与Server才能相互确认双向连接，实现双工数据传输。

为什么要四次挥手?

答：“三次握手”的第二次握手发送SYN+ACK回应第一次握手的SYN，但是“四次挥手”的第二次挥手只能发送ACK回应第一次挥手的FIN，因为此时Server可能还有数据传输给Client，所以Server传输数据完成后才能发起第三次挥手发送FIN给Client，等待Client的第四次挥手ACK。

三次握手与四次挥手？

答：

TCP的三次握手和四次挥手实质就是TCP通信的连接和断开。

三次握手：为了对每次发送的数据量进行跟踪与协商，确保数据段的发送和接收同步，根据所接收到的数据量而确认数据发送、接收完毕后何时撤消联系，并建立虚连接。

四次挥手：即终止TCP连接，就是指断开一个TCP连接时，需要客户端和服务端**总共发送4个包**以确认连接的断开。

TCP协议位于传输层，作用是提供可靠的字节流服务，为了准确无误地将数据送达目的地，TCP协议采纳三次握手策略。

三次握手原理：

第1次握手：客户端发送一个带有SYN（synchronize）标志的数据包给服务端；

第2次握手：服务端接收成功后，回传一个带有SYN/ACK标志的数据包传递确认信息，表示我收到了；

第3次握手：客户端再回传一个带有ACK标志的数据包，表示我知道了，握手结束。

其中：SYN标志位数值置1，表示建立TCP连接；ACK标志表示验证字段。

HTTP的头部信息包含哪些？

答：HTTP报文中的头部信息包含了关于请求或响应的元数据。HTTP头部信息由一系列字段-值对组成，每个字段-值对之间使用冒号分割。常见的有：

- 1) Content-Type: 指定请求或响应的媒体类型；
- 2) Content-Length: 指定请求或响应主体的长度，以字节为单位；
- 3) Accept: 指定客户端可以接受的响应内容类型；
- 4) Accept-Language: 指定客户端可以接受的语言类型；
- 5) User-Agent: 指定客户端用户代理的相关信息；
- 6) Host: 指定服务器的主机名和端口号；
- 7) Cookie: 用于在客户端和服务器之间传递和存储会话信息的数据；
- 8) Location: 用于指定重定向的目标URL；

HTTP状态码

答：状态码是用来表示HTTP请求是否已经完成，分为五大类型

- 1XX: 表示请求已经收到，正在处理；
- 2XX: 表示请求已经成功被服务器接收、理解和处理；
- 3XX: 重定向状态码，表示客户端需要采取进一步的操作才能完成请求；
- 4XX: 表示客户端发送的请求有误；
 - 400: 指出客户端请求中的语法错误；
 - 401: 示发送的请求需要认证；
 - 403: 表明对请求资源的访问被服务器拒绝了；
 - 404: 表明服务器上无法找到指定的资源

5XX: 表示服务器在处理请求时发生了错误 ;

- 500: 表明服务器端在执行请求时发生了错误;
- 502: 表明服务器网关错误;
- 503: 表明服务器上暂时处于超负载或者正在进行停机维护, 现在无法处理请求;

http请求的方式有哪些?

答: http最大的作用就是确定了请求和响应数据的格式。常见的请求方式:

- 1) get方法
发送一个请求来取得服务器上的某一资源。
- 2) post方法
向url指定的资源提交数据或附加新的数据。
- 3) put方法
put方法跟post方法很像, 也是向服务器提交数据, 但是put方法指向了资源在服务器上的位置, 而post方法没有。
- 4) head方法
只请求页面的首部。
- 5) delete方法
删除服务器上的某资源。

https和http的区别

答: 1) https的端口是443, 而http的端口是80, 且两者的连接方式不同; 2) http传输是明文的, 而https是用ssl进行加密的, https的安全性更高; 3) https是需要申请证书的, 而http不需要。

说说对http的理解?

答: HTTP是一种用于在web上进行数据传输的协议。它是一种无状态的、可靠的、应用层协议, 通过客户端和服务端之间的请求-响应模型进行通信。它的基本工作原理如下: 1) 客户端发起请求; 2) 服务器处理请求; 3) 服务器发送响应; 4) 客户端处理响应

数据链路层、网络层、传输层、应用层、物理层的协议

答: 数据链路层: PPP、PPTP
网络层: IP、RIP
传输层: TCP、UDP
应用层: HTTP、HTTPS、RTSP、FTP
物理层: 物理线路, 光纤, 无线电

OSI七层模型和TCP/IP五层模型

答：OSI定义了网络互联的七层模型（物理层、数据链路层、网络层、传输层、会话层、表示层、应用层）

应用层：为应用程序或用户请求提供各种请求服务。

表示层：数据编码、格式转换、数据加密

会话层：创建、管理和维护会话

传输层：数据通信

网络层：IP选址及路由选择

数据链路层：提供介质访问和链路管理

物理层：管理通信设备和网络媒体之间的互联互通

ping命令是基于ICMP协议

TCP的三次握手和四次挥手？

答：TCP是一种面向连接的可靠传输协议。在建立连接和断开连接的时候，TCP使用三次握手和四次挥手来进行通信。

三次握手：

第一次（SYN）：客户端发送一个带有syn标志的连接请求报文段给服务器，并指定初始序列号；

第二次（SYN+ACK）：服务器收到客户端的连接请求报文段后，如果同意建立连接，就会发送一个带有SYN/ACK标志的报文段作为应答。同时，服务器也会为自己分配一个初始序列号；

第三次（ACK）：客户端收到服务器的应答后，会发送一个带有ACK标志的报文段给服务器，确认建立连接。同时，报文段中也会指定客户端下一个要发送的数据的序列号。

四次挥手：

第一次（FIN）：当客户端需要断开连接时，发送一个带有FIN标志的报文段给服务器，表示客户端没有数据要发送了；

第二次（ACK）：服务器收到客户端的断开请求后，发送一个带有ACK标志的报文段作为应答，表示服务器收到了客户端的断开请求；

第三次（FIN）：如果服务器也没有数据需要发送，就会发送一个带有ACK标志的报文段给客户端，表示服务器同意断开连接；

第四次（ACK）：客户端收到服务器的断开请求后，发送一个带有ACK标志的报文段作为应答，表示客户端统一断开连接。

为什么TCP要建立三次握手？

答：三个方面进行回答：

1) TCP协议，是一种可靠的，基于字节流的，面向连接的传输层协议。它的可靠性体现在TCP协议通信的双方的数据传输是稳定的，即便是在网络不好的情况下，TCP都能保证数据传输到目标端，而这个可靠性，是基于数据包的确认机制来实现的；

2) TCP协议双方的数据传输是通过节流来实现传输的，是面向连接，在进行数据通信之前，需要建立一个可靠的连接，TCP使用三次握手来实现连接的建立，所谓的三次握手，就是通信双方一共需要发送三次请求，才能确保这个连接的建立。具体是：客户端向服务端发送连接请求，并携带同步序列号SYN；服务端收到请求后，发送SYN和ACK，这里的SYN表示的是服务端的同步序列号，ACK表示对前面收到请求的一个确认；客户端收到服务端请求后，再次发送ACK，这个ACK是针对服务端连接的一个确认；

3) 之所以TCP要涉及三次握手，我认为有三个方面原因：

第一，TCP是一个可靠性通信协议，所以TCP协议的通信双方都必须要维护一个序列号，去标记已经发送出去的数据包，哪些是已经被对方签收的，而三次握手

就是通信双方相互告知的序列号的一个起始值，为了确保这个序列号能收到，所以双方都需要一个确认的动作；第二，TCP协议需要在一个不可靠的网络环境中进行可靠的传输，意味着通信双方必须要通过某种手段来实现一个可靠的数据传输通道，而三次通信是建立这样的一个通道的最小值，当然还可以四次、五次，只是没有必要去浪费这个数据资源；第三，防止历史的重复连接初始化造成的混乱问题，比如网络比较差的情况下，客户端连续多次发送建立连接请求，假设只有两次握手，那么服务端只能选择接收或者拒绝这个连接，但是服务端并不知道这次请求是不是之前因为网络堵塞而过期的请求，也就是说服务端不知道当前客户端的连接是有效的还是无效的

说说cookie和session的区别

答：cookie是客户端浏览器用来保存服务端数据的一种机制，当我们通过浏览器去进行访问的时候，服务器可以把某一些状态数据以key-value的形式写入到cookie里面，存储到客户端浏览器；然后客户端下一次再访问服务器的时候，我们可以携带着一些状态数据发送到服务器端，服务器端可以根据cookie里面携带的内容去识别使用者；

而session表示一个会话，他是属于服务器端的一个容器对象，默认情况下，针对每一个浏览器的请求，servlet容器都会分配一个session对象，session本质上我们可以认为它是一个concurrentHashMap，它可以用来存储当前会话产生的一些状态数据。我们都知道，http协议本身是一个无状态的协议，也就是说服务器端并不知道客户端发送过来的多次请求是否属于同一个用户,所以session是用来

弥补http无状态的一个不足。简单来说，服务器端可以利用session来存储客户端在同一个会话里面产生的多次请求的一个记录。那么基于服务器端的session的存储机制，再结合客户端的cookie机制，我们就可以去实现一个有状态的http协议，它的具体工作原理非常简单，首先，客户端第一次访问服务器端的时候

服务器端会针对这次请求创建一个会话，并且生成一个唯一的sessionid来标注这个会话，然后服务器端把这个sessionid写入到客户端浏览器的cookie里面，用来去实现客户端状态的一个保存，那么在后续的一个请求里面，每次都会携带sessionid，服务器端就可以根据sessionid来识别当前的这个会话的状态。所以总的来看cookie是客户端的存储机制，而session是服务器端的存储机制。

JVM

JVM内存区域总共分为两种类型

- 1) 线程私有区域：程序计数器、本地方法栈和虚拟机栈
- 2) 线程共享区域：堆和方法区

特征

线程私有区域：依赖用户的线程创建而创建、销毁而销毁，因用户每次访问都会独立开启一个线程，跟本地的线程相对应

线程共享区域：它是随着虚拟机的开启而创建，关闭而销毁。

JVM中类加载机制

JVM类加载器机制分为五个部分：加载、验证、准备、解析、初始化。

类加载器

JVM提供三种类加载器：

启动类加载器

负责加载JAVA_HOME\lib目录中，或通过-Xbootclasspath参数指定路径中的，且被虚拟机认可的类

扩展类加载器

负责加载JAVA_HOME\lib\ext目录中的，或通过java.ext.dirs系统变量指定路径中的类库。

应用程序类加载器

负责加载用户路径上的类库

什么是双亲委派机制？它有什么作用？

双亲委派机制的意思是类加载器，在加载之前，都会委派给它的父加载器进行加载，这样一层层向上传递，直到祖先们都无法胜任，它才会真正的加载；

好处：

- 1) 可以避免类的重复加载；
- 2) 保证JAVA程序安全稳定运行，JAVA核心API定义类型不会被随意替换。

如何理解局部变量、成员变量以及静态变量？

成员变量在堆上、局部变量在栈上、静态变量在方法区

G1和CMS收集器的区别

答：区别一：使用范围不一样

CMS是老年代的收集器，可以配合新生代的serial收集器一起使用；G1收集器收集范围是老年代和新生代，不需要结合其他收集器使用；

区别二：STW的时间

CMS收集器以最小的停顿时间为目标的收集器。

G1收集器可预测垃圾回收的停顿时间（建立可预测的停顿时间模型）

区别三：垃圾碎片

CMS收集器是使用“标记-清除”算法进行的垃圾回收，容易产生内存碎片G1收集器使用的是“标记-整理”算法，进行了空间整合，没有内存空间碎片。

区别四：垃圾回收的过程不一样

注意这两个收集器第四阶段得不同，不清楚可以看深入理解jvm这本书区别

五：CMS会产生浮动垃圾

CMS产生浮动垃圾过多时会退化为serialold，效率低，而G1没有浮动垃圾，

jvm中堆和栈的区别？

答：1) 各司其职

最主要的区别就是栈内存用来存储局部变量和方法调用；而堆内存用来存储java中的对象。无论是成员变量，局部变量，还是类变量，它们指向的对象都存储在

堆内存中。

2) 栈是线程私有的，堆是线程共享的；

3) 栈的内存要远远小于堆内存；

java对象的存储

答：在java中，每个对象都被存储在了堆中。堆是一种运行时数据区，用于存储所有的对象，当我们创建一个对象时，它会在堆中分配一段连续的内存空间，用于存储对象的属性值。

垃圾回收

1) 什么是垃圾回收

垃圾回收是指不定时去堆内存中清理不可达对象。

2) 为什么需要垃圾回收

如果不进行垃圾回收，内存迟早都会被消耗空，因为我们在不断的分配内存空间而不进行回收。除非内存无限大，我们可以任意的分配而不回收，但是事实并非如此。所以垃圾回收是必须的；

3) 垃圾识别机制

引用计数法

它是判断对象是否存活的算法之一：它给每个对象加一个引用计数器，每当有一个地方引用它时，计数器值就加1；当引用失效时，计数器就减1；任何时刻计数器为0的对象就是不可能被使用的，即被垃圾回收器回收。

缺点：需要单独的字段存储计数器，增加了存储空间开销；无法解决对象间互相循环引用的问题，可能会造成内存泄漏；

可达性分析算法

目前主流使用的都是可达性分析算法来判断对象是否存活。算法基本思路：以“GCRoots”作为对象的起点，从节点开始向下搜索，搜索所走过的路径成为引用链。当一个对象到GCRoots没有任何引用链相连时，则证明此对象是不可用的。

哪些对象可作为GCRoots？

虚拟机栈中引用的对象；方法区中类静态属性引用的对象；方法区中常量引用的对象；本地方法栈中引用的对象；活跃线程的引用对象。

4) 四种垃圾回收算法

标记清除法

算法分为标记和清理两个步骤，首先标记出所有需要回收的对象，在标记完成后统一回收所有被标记的对象；

效率不高，会产生内存碎片

适用于存活对象占多数的情况

标记整理算法

标记过程和标记清除算法一样，而后面的不一样，它是让所有存活对象都向一端移动，然后直接清理掉端边界以外的内存

不会产生内存碎片

复制算法

将可用内存划分为大小相等的两块，每次只使用其中的一块。当这一块内存用完后，就将还存活的对象复制到另一块去，然后再把已使用过的内存空间一次性清理掉；

缺点：

可用内存缩小为了原来的一半；算法执行效率高，适用于存活对象占少数的情况。

分代收集算法

当前大多数垃圾收集都采用分代收集算法，这种算法并没有什么新的思路，只是根据对象存活周期的不同将内存划分为几块，每一块使用不同的上述算法去收集。

在jdk8以前分为三代：年轻代、老年代、永久代。在jdk8以后取消了永久代的说法，而是元空间取而代之。

在新生代，每次垃圾收集器都发现有大批对象死去，只有少量存活，采用复制算法，只需要付出少量存活对象的复制成本就可以完成收集。

而老年代中，因为对象存活率高，没有额外的空间对它进行分配担保，就必须“标记-清除-压缩”算法进行回收。

5) MinorGC和FullGC的区别

新生代GC（MinorGC）：只发生在新生代的垃圾收集动作，因为java对象大多具备朝生夕灭的特性，所以MinorGC非常频繁，一般回收速度也比较快；老年代GC

（MajorGC/FullGc）：指发生在老年代的GC，出现了MajorGC，经常会伴随至少一次的MinorGC。MajorGC的速度一般会比MinorGC慢10倍以上。

JVM组成部分？

答：类加载器、运行时数据区、执行引擎、本地库接口。其中运行时数据区又由：程序计数器、虚拟机栈、本地方法栈、java堆和方法区组成。

JVM运行时数据区？

答：运行时数据区一共包括五个部分，分别是程序计数器、虚拟机栈、本地方法栈、java堆和方法区。其中程序计数器、虚拟机栈和本地方法栈是线程私有的，java堆和方法区是线程共享的。

（栈帧：栈的存储单位为栈帧，线程上正在执行的一个方法对应着一个栈帧，栈帧是一个内存区块，是一个数据集，维系着方法执行过程中的各种数据信息。

每个栈帧包括五个部分：分别包括局部变量表、操作数栈、动态链表、方法返回地址和一些附加信息）。

JMM是什么？

答：JMM是Java内存模型的简称，它定义了java程序中多线程访问共享内存的行为规范。JVM规定了线程如何与内存交互，以及如何通过读取和写入共享变量来实现线程之间的同步。JMM确保了在多线程并发执行的情况下，程序的执行结果是可预测的。它提供了一组底层内存操作指令，并规定了这些操作的执行顺序和可见性

JVM常见命令

答：1) javac：用于编译java源代码文件为字节码文件；

- 2) javap: 用于反编译字节码文件, 将.class文件转换成可读的JAVA源代码; 3) Javah: 用于生成C头文件;
- 4) Jps: 用于查看正在运行的java进程和它们的进程id。
- 5) Jstat: 用于监控和收集java虚拟机的统计信息;
- 6) Jmap: 用于生成java堆内存的转存文件;
- 7) Jstack: 用于生成java线程的堆栈信息。

GC是什么? 为什么要有GC?

答: GC (GarbageCollection), 是垃圾收集的意思。内存处理是编程人员容易出现问题的地方, 忘记或者错误的内存回收会导致程序或者系统的不稳定甚至崩溃, JAVA提供的GC功能可以自动检测对象是否超过作用域从而达到自动回收内存的目的, 这样一来, 程序员就不用担心内存管理, 因为垃圾收集器会自动进行管理。

垃圾回收器的基本原理是什么? 垃圾回收器可以马上回收内存吗? 有什么办法

主动通知虚拟机进行垃圾回收?

答: 对于GC来说, 当程序员创建对象时, GC就开始监控这个对象的地址、大小以及使用情况。通常GC采用可达性分析算法记录和管理堆中的所有对象通过这种方式确定哪些对象是“可达的”, 哪些对象是“不可达的”。当GC确定一些对象为“不可达”时, GC就有责任回收这些内存空间;
可以
程序员可以手动执行System.gc(), 通知GC运行, 但是JAVA语言规范并不保证GC一定会执行。

锁

乐观锁与悲观锁

它们的最终目的都是为了保证线程安全, 避免在并发场景下的资源竞争问题;
乐观锁 (多读): 对线程的安全问题总是持有乐观态度, 认为每次访问数据时都不会有人来修改数据, 因此也不加锁, 只是在提交操作时检查是否违反数据完整性。

悲观锁 (多写): 对线程的安全问题总是持有悲观态度, 认为每次访问时都会有人来修改数据, 于是每次操作前都会加锁。

乐观锁适合于多读场景, 悲观锁适合于多写场景

公平锁与非公平锁

答：公平锁：按照线程等待顺序获取锁，一般将获取锁失败的线程放入等待队列中，每次从FIFO队列的队头取出线程获取锁。这种方式会造成性能低下，大量的时间花费在线程调度上。

非公平锁：不管等待顺序，每个线程获取锁的概率都是相等的，优点是提高了响应速度，不用把大量时间花费在线程调度上，而是花费在执行代码上。

共享锁与独占锁

答：独占锁：一次只允许一个线程访问资源，也叫互斥锁；

共享锁：允许有多个线程并发访问共享资源。读写锁就是共享锁。

什么是可重入锁？

答：可重入锁又名递归锁

是指在同一个线程外层方法获取锁的时候，再进入该线程的内层方法会自动获取锁(前提，锁对象得是同一个对象)，不会因为之前已经获取过还没释放而阻塞。

Java中ReentrantLock和Synchronized都是可重入锁，可重入锁的一个优点是可一定程度避免死锁。

锁机制

答：java中的锁机制是用来处理多线程并发情况下数据的一致性的。在我们操作一个对象或者调用一个方法前加锁，这样当其他线程也对该对象和方法进行访问时就需要获得锁，如果该锁被其他线程持有，那么该线程则进入阻塞队列等待获得锁。这样就保证了在同一时间只有一个线程在对该对象进行操作。

java中的锁，从不同角度有好几种划分，比如：（悲观锁、乐观锁）、（公平锁、非公平锁）、（共享锁、独占锁）等。

什么是死锁？

答：两个或两个以上的线程都在等待对方执行完毕才能继续往下执行的时候就发生了死锁，结果就是这些线程都陷入了无限的等待中。

产生死锁的条件

答：假设有两个共享资源A和B、两个线程1和2。线程1持有资源A，线程2持有资源B。但是现在线程1又想访问资源B，线程2想访问资源A。都在等待对方释放锁，就产生了死循环，死锁。

产生死锁的四个必要条件：

- 1) 互斥条件：一个资源每次只能被一个进程或者线程使用；
- 2) 请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放；
- 3) 不可剥夺条件：进程已获得的资源，在未使用完之前，不能强行剥夺；
- 4) 循环等待条件：若干进程之间形成一种头尾相连的循环等待资源关系

获取锁失败后当前线程会怎样

答：当前线程等待或者抛出异常

除了锁的方式，还有哪些方式实现并发？

答：1) 局部变量；2) 不可变对象；3) ThreadLocal：当多个线程去访问ThreadLocal对象是，都会单独的为访问的线程提供一个该对象的副本。即每个对象只会被一个线程操作，自然无并发问题。4) CAS

SSM

说说对springMVC的理解

答：关于这个问题我将从几个方面来回答。

首先SpringMVC是属于SpringFramework生态里面的一个模块，它是在servlet的基础上构建并且使用了MVC模式，设计的一个web框架，它的主要目的是为了简化传统的servlet+JSP模式下的web开发方式。

其次springMVC的整个架构设计是对javaweb里面的mvc框架模式做了一些增强和扩展，主要体现的几个方面：第一个是把传统的MVC框架里面的controller控制器做了拆分，分成了前端控制器DispatcherServlet和后端控制器Controller;第二个是把model模型拆分成业务层的service和数据访问层的repository；第三个，在视图层面，可以支持不同的视图，比如说freemark、velocity、JSP等等。所以springMVC天生就是为了MVC模式而设计的，因此在开发MVC应用的时候会更加方便灵活。

最后再说一下SpringMVC的整体工作流程，浏览器的请求首先会去经过Spring MVC里面的核心控制器DispatcherServlet，它主要是把请求分发到对应的Controller里面，而Controller里面处理完业务逻辑之后，会返回一个ModelAndView，然后DispatcherServlet会去寻找一个或者多个viewResolver的视图解析器，找到modelAndView指定的视图，并且把数据展示到客户端。

SpringMVC的处理流程？

- 答：1) 客户端发送请求到前端控制器（dispatcherServlet）；
- 2) 前端控制器请求HandlerMapping查找Handler，可以根据xml配置、注解进行查找；
- 3) 处理器映射器根据HandlerMapping向前端控制器返回Handler； 4) 前端控制器调用处理器适配器去执行Handler；
- 5) 处理器配置器执行Handler；
- 6) Handler执行完成后给适配器返回ModelAndView；
- 7) 处理器适配器向前端控制器返回ModelAndView；
- 8) 前端控制器请求视图解析器去进行视图解析，根据逻辑视图来解析真正的视图；
- 9) 视图解析器向前端返回view；
- 10) 前端控制器进行视图渲染，就是将在ModelAndView对象中的模型数据填充到request域；
- 11) 前端控制器向用户响应结果；

@Data注解的作用？

答：@Data注解是lombok.jar包下的注解，主要作用是提高代码的简洁，使用这个注解可以省去实体类中大量的get()、set()、toString()等方法。

@RequestBody注解的作用？

答：主要是用来接收前端传递给后端的json字符串中的数据（请求体中的数据）；一个请求，只有一个RequestBody；一个请求，可以有多个RequestParam。

MVC模型介绍一下。

答：实现步骤：分为三级。

M(Model):持久层，代码与数据库进行交互的代码（Mybatis-dao层）

V (View)：视图层，一般是指用户看到的内容（页面）

C (Cotrol)：控制层，完成某项业务的具体操作过程（Controller层-Service层）

Spring三件套的框架？

答：Spring三件套指的是Spring框架的三个核心模块，分别是SpringCore、Spring AOP和SpringMVC。

SpringCore: 提供了IoC容器，用于对象之间的解耦，通过容器自动将对象之间的依赖注入。同时，**SpringCore**还提供了基于注解的**SpringBean**的支持。**SpringAOP:** 提供了基于AOP的编程方式，能够在不修改源代码的情况下，通过代理机制对对象进行增强，比如添加事务、日志、安全检查等功能。

SpringMVC: 是**Spring**框架的Web模块，提供了MVC模式的支持，用于处理Web请求和响应。**SpringMVC**通过DispatcherServlet、HandlerMapping、Controller和ViewResolver等组件构成了完整的MVC模式的实现。

AOP面向切面编程的好处？以及怎么实现？

答：就是将通用的、与主业务逻辑无关的代码（比如事务、日志、缓存等），利用AOP容器的功能将切面织入到主业务逻辑中。

利用AOP可以对业务逻辑的各个部分进行隔离，从而使业务逻辑各部分之间的耦合度降低，提高程序的可重用性，同时提高了开发的效率。

有两种实现方式，第一种是通过原生jdk实现，首先要定义接口，实现一个切面代理；第二种通过cglib切面来实现。这两种方式相同点是都使用了代理模式，不同点是前者使用了接口代理，后者使用了继承代理。

Springboot的优缺点？

答：优点：1）可以快速整合第三方框架，无需配置文件；2）内嵌了servlet容器，降低了对环境的要求；3）减少了大量的开发时间，并提高了生产力；4）避免了编写大量的样板代码，注释和XML配置；5）提供了很多插件来开发和测试。缺点：1）迭代快；2）封装太深，内部原理复杂，不容易精通。

spring事务相关的注解？有哪些参数？

答：@Transactional

参数：1）propagation：事务传播行为由传播属性来指定；

2）ioslation：事务隔离级别

3）Timeout:超时时间

4）readOnly：是否只读

5）rollbackFor：回滚

6）noRollbackFor：不回滚

springboot和springmvc的区别

答：1）含义不同

springboot：SpringBoot是一个自动化配置的工具。

springmvc：SpringMVC是一个web框架。

2）配置不同

springboot：SpringBoot采用约定大于配置的方式，通过其自动配置功能自动处理配置，同时内置服务器，打开就可以直接用。

springmvc: 此框架需要大量配置，例如DispatcherServlet配置和ViewResolver配置。需要手动配置xml文件，同时需要配置Tomcat服务器。

3) 开发时间不同

springboot: SpringBoot有助于减少开发时间，因为所有与依赖关系相关的任务都会得到处理。

springmvc: 与SpringBoot相比，开发所需的时间更多，因为开发人员需要花时间添加所需的依赖项。

4) 生产力不同

springboot: 由于开发时间更短，生产力提高。

springmvc: 生产力降低，因为需要了解依赖性附加组件。

5) 是否提供批处理功能

springboot: 它提供强大的批处理。

springmvc: 它不提供强大的批处理。

6) 是否需要部署描述符

springboot: 不需要部署描述符。

springmvc: 需要部署描述符。

介绍下springmvc

答: springmvc是spring的web模块，能够处理web的请求和响应。MVC的英文全称是modelviewcontroller，它把系统模型分为模型，视图和控制器三个部分。Model是程序逻辑部分；view是页面显示部分；controller是前后端交互部分。

springboot的常用注解?

答: 1) 启动类注解@SpringBootApplication，这是springboot最核心的注解，用于springboot的主类，标识这是一个springboot应用，用来开启springboot的各项能力。

2) @Controller: 表示一个控制器类，用于处理HTTP请求和响应。通常与@RequestMapping注解一起使用；

3) @RestController: 与@Controller类似，但是会自动将方法的返回值转换成json或者XML格式。

4) @RequestMapping: 用于将HTTP请求映射到控制器的特定方法上，可以用来定义请求的路径和请求方法；

5) @Autowired: 用于自动装配依赖关系，将需要的Bean自动注入到当前类中。

Spring,springboot,springmvc都是干什么的?

答: spring提供了一套开发企业级java应用程序的综合解决方案，包括依赖注入、面向切面编程等功能；

springmvc是基于spring框架的mvc设计模式，用于开发web应用程序，提供了统一的请求处理流程和扩展性；

springboot是spring框架的扩展，简化了spring应用程序的开发和部署，通过自

动配置和快速构建工具提供了开箱即用的方式来构建独立的、可扩展性的spring应用程序。

Redis

Redis为什么会这么快？

答：1) Redis是纯内存操作；
2) Redis是单线程，从而避开了多线程中上下文频繁切换的操作；3)
Redis数据结构简单、对数据的操作也比较简单；
4) 使用多路I/O复用模型，非阻塞I/O；

介绍一下什么是阻塞I/O，什么是非阻塞I/O，什么是I/O多路复用？

答：阻塞I/O：进程发起read后，如果kernel未准备好数据，进程就会被阻塞，进入等待阶段；等待kernel将数据准备好，才会返回数据，解除阻塞；

非阻塞I/O：进程发起read请求，kernel未准备好数据，直接返回一个error，进程无需阻塞等待；进程会轮询发送read请求，如此往复；一直到kernel准备好数据，才把数据拷贝并返回给进程，结束轮询；

I/O多路复用：进程调用select/poll/epoll，select/poll/epoll会将进程block起来；kernel会‘监视’所有select负责的socket；

当任何一个socket准备好数据，select就会返回（也就是图中的returnmedable）；进程调用read（图中第二次systemcall），将数据从kernel拷贝到用户进程；总结：所以，I/O多路复用的特点是通过一种机制一个进程能同时等待多个文件描述符，而这些文件描述符（套接字描述符）其中的任意一个进入就绪状态，select()函数就可以返回。

redis的持久化机制？

答：redis中有两种持久化机制，分别是RDB和AOF两种方式。两种持久化方式可以单独使用其中一种，也可以将这两种方式结合使用。

RDB称为数据快照，根据指定的规则定期将内存中的数据存储到硬盘上；它是redis默认的持久化方式；

AOF是追加文件。redis处理的每一个写命令都会记录在AOF文件，在固定时间会进行压缩。

引入缓存的好处？

答：redis不仅性能很高，还能提供很多友好的数据类型，可以很好的满足我们的业务需求。

如何提高缓存的利用率？

答：让缓存中只保留最近访问的热数据，具体这么来优化：
写请求依旧只写数据库；读请求先读缓存，如果缓存不存在，再从数据库中读取，并重建缓存；同时，写入缓存中的数据，都设置失效时间；
这样一来，缓存中不经常访问的数据，随着时间的推移，都会逐渐过期淘汰掉；
最终缓存中保留的，都是经常被访问的热数据，缓存利用率得以最大化。

如何保证数据库和缓存中的数据一致性？

答：延迟双删：先删除缓存数据，再写数据库，休眠一会后再次删除该缓存数据。
这么做的目的，就是确保读请求结束，写请求可以删除读请求造成的缓存脏数据。

谈谈你对redis的理解？

答：是一个基于key-value存储结构的Nosql开源内存数据库，它提供了5种常用的数据类型，有：String、Hash、Zset、Set、List，针对不同的结构可以解决不同场景的问题，因此它可以去覆盖应用开发里面的大部分的业务场景，如top10问题、好友关注列表和热点话题等等。

其次，因为redis是一个基于内存的存储，在数据结构上做了大量的一些优化，所以IO性能比较好。在实际开发中，我们会把它用在应用和数据库之间的一个分布式缓存中间件。

并且呢，他又是一个非关系型数据库的存储，他不存在表之间的关联查询等一些问题，所以呢它可以很好的去提升应用程序的数据IO效率。

最后对于企业级开发来说，它又提供了主从复制+哨兵，以及集群的方式去实现高可用。

分布式锁的三种实现方式是什么？

答：1）基于数据库实现分布式锁（乐观锁、悲观锁）；

2）基于缓存（redis等）实现分布式锁；

3）基于zookeeper实现分布式锁；

从性能从高到低来看，缓存方式>Zookeeper方式>=数据库方式；

redis中的LUA脚本如何保证原子性的？

答：在数据库中事务的ACID中原子性指的是“要么都成功要么都失败”，而在并发编程中的原子性指的是“操作不可拆分、不被中断”。Redis既是一个数据库，又是一个支持并发编程的系统，所以它的原子性有两种。LUA脚本中的原子性指的是在并发编程中的原子性。

在Redis中，Lua脚本能够保证原子性的主要原因还是Redis采用了单线程执行模型。也就是说，当Redis执行Lua脚本时，Redis会把Lua脚本作为一个整体

并把它当作一个任务加入到一个队列中，然后单线程按照队列的顺序依次执行这些任务，在执行过程中Lua脚本是会被其他命令或请求打断，因此可以保证每个任务的执行都是原子性的。

redis中的LUA脚本有什么用

答：RedisLua脚本是一种在Redis服务器端执行的脚本语言，它可以非常高效地操作Redis的数据结构和执行复杂的逻辑。RedisLua脚本具有以下几个主要的用途：

- 1.原子性操作：RedisLua脚本的执行是原子性的，即脚本中的操作会在一个事务中进行，保证了多个命令的连续执行不会被其他客户端或线程中断，避免了并发操作的竞争条件。
- 2.复杂逻辑处理：RedisLua脚本可以执行复杂的逻辑处理，包括判断、循环、条件语句等。这使得开发者可以在脚本中灵活地进行一些数据处理、计算和业务逻辑的操作，避免了多次往返的网络请求。
- 3.原子性操作的封装：通过将多个操作封装在一个Lua脚本中，可以确保这些操作的原子性执行，避免了因为网络延迟或其他原因而导致的操作不一致问题。
- 4.服务器端运算：通过RedisLua脚本，可以将一些运算操作放到服务器端进行，减轻了客户端的负担和网络传输的开销，提高了性能和效率。
- 5.扩展功能：RedisLua脚本提供了一些Redis本身没有的扩展功能，例如自定义命令、封装复杂的查询、定义定时任务等。这些功能可以根据实际需求来扩展和定制Redis的功能。

总的来说，RedisLua脚本提供了一种简洁、高效、安全的方式来处理Redis的数据和逻辑操作。通过合理使用Lua脚本，可以在保证原子性的同时，提高性能，减少网络开销，并且可以进行一些复杂的操作和扩展功能。

redis集群的模式有哪些？

答：1）主从复制；2）哨兵模式；3）cluster集群
（哨兵模式主要执行三件事：监控、提醒、自+动故障迁移）
（集群是什么？

简单的理解就是多个redis服务器进行完成同一个任务或者在并发状态下可以分担或者代替master成为新的master继续进行一个工作的任务或者保持数据的完整性、一致性）。

redis的淘汰机制？

答：有六种淘汰机制

- 1) noeviction（默认策略）：对于写请求不再提供服务，直接返回错误；2) Allkeys-lru：从所有key中使用LRU算法进行淘汰（LRU算法：即最近最少使用算法）3) volatile-lru：从设置了过期时间的key中使用LRU算法进行淘汰4) allkeys-random：从所有key中随机淘汰数据

- 5) volatile-random: 从设置了过期时间的key中随机淘汰
- 6) volatile-ttl: 在设置了过期时间的key中, 淘汰过期时间剩余最短的

说说对缓存雪崩和缓存穿透、缓存击穿的理解

答: 缓存雪崩, 就是存储在缓存里面的大量数据在同一个时刻全部过期, 原本缓存组件能够抗住的大部分流量, 全部请求到了数据库, 从而导致数据库压力增加, 造成数据库服务器崩溃的一种现象。导致缓存雪崩的原因呢, 我认为有几个方面, 第一个是缓存中间件宕机, 其次就是缓存中大部分key都设置了相同的过期时间, 导致同一时刻这些key都过期了对于这种情况可以在失效时间里面去增加1-5分钟的随机值, 从而去避免同时失效的一个问题。

缓存穿透, 表示的是短时间内, 有大量的不存在的key, 请求到应用程序里面, 而这些不存在的key在缓存里面又找不到, 从而就导致全部请求全部穿透到了数据库, 造成数据库的压力增加。有两个方法可以解决, 第一个是把无效的key保存到redis里面, 并且设置一个特殊的值, 比如说像“null”字符串, 这样的话, 下次再来访问时, 就不会来查询数据库了。但是如果攻击者不断的用随机的不存在的key来访问, 也还是会存在相同的问题, 这时可以使用布隆过滤器来实现, 那么在系统启动的时候, 我们可以把目标数据全部缓存到布隆过滤器里面。当攻击者使用不存在的key来请求的时候, 先到布隆过滤器里面去进行查询, 如果不存在, 就意味着这个key在数据库中也肯定不存在, 所以这个时候就不会去访问数据库。布隆过滤器还有个好处, 就是他采用位图来进行存储, 所以它的占用内存空间是很小的。

缓存击穿是指某个热点数据缓存过期后, 恰好有大量并发请求同时访问这个热点数据, 导致每个请求都直接访问数据库, 增加了数据库的负载。解决方案: 使用互斥锁或分布式锁对热点数据进行加锁, 当有请求访问这个热点数据时, 只允许一个请求去更新缓存或查询数据库, 其他请求等待缓存更新后再获取数据。

缓存穿透与缓存击穿的区别?

答: 缓存穿透就是访问一个不存在的key, 缓存不起作用, 请求会穿透到数据库中, 流量大时数据库会挂掉。解决方法是布隆过滤器。

缓存击穿是访问一个存在的key, 并且针对的是某个具体的热点数据, 在缓存过期的那一刻, 有大量并发请求过来, 这些请求都会击穿到数据库, 造成数据库压力骤增, 甚至崩溃。解决方式可以设置热点数据永不过期

(缓存穿透是指缓存和数据库中都没有的数据;

缓存击穿是指缓存中没有但是数据库中有的数据(一般是缓存时间到))

redis哨兵机制和集群有什么区别?

答: 因为redis集群有两种, 一种是主从复制, 一种是rediscluster。1) Redis哨兵集群是基于主从复制来实现的, 所以它可以实现读写分离, 分担redis读操作的一个压力; 而rediscluster集群里面的slave节点呢,

只是实现冷备的一个机制，它只有在master宕机后才会工作；

2) 第二个，redis哨兵集群无法在线扩容，所以它的并发压力受限于单个服务器资源的一个配置，Rediscluster提供了基于slot槽的数据分片的机制，它可以实现在线扩容，去提升读写的一个性能；

3) 第三个，从集群的架构来说，redis哨兵集群，是一主多从；而redis cluster是一个多主多从的机制

RDB和AOF的实现原理和优缺点？

答：首先，redis本身是一个key-value结构的内存数据库，它的设计者为了避免其故障导致数据丢失的一个问题，提供了RDB和AOF两种持久化的一个机制。RDB是通过快照的方式来实现持久化的，就是说会根据快照的一个触发条件，把内存里面的数据快照写入到磁盘里面，以二进制的压缩文件的方式进行存储。

AOF是一种近乎实时的方式，把redisserver执行的事务命令进行追加存储，简单来说就是客户在执行一个数据变更的操作，redis就会把这样的命令追加到AOF的缓冲区里面，然后再把缓冲区里面的数据写入到磁盘的AOF文件里面，至于什么时候真正把数据持久化到磁盘的AOF文件里面，是根据磁盘的刷盘策略来决定的。

优缺点：

- 1) RDB是每隔一段时间会触发持久化，因此数据安全性低；AOF可以做到实时持久化，数据安全性比较高；
- 2) RDB文件默认采用压缩的方式持久化，AOF存储的是执行指令，所以RDB在数据恢复的时候性能比AOF要好；

Redis如何尽可能多的存储数据？

答：1) 数据分片：将数据分散存储到多个redis节点上，可以通过对数据的key进行哈希运算，将不同的key映射到不同的redis节点上。这样可以将存储负载均衡到多个节点上，以实现多存储。

2) 使用rediscluster：通过将数据分布到多个节点上，可以扩展存储容量，并提高整体的读写性能；

3) 利用持久化和备份：可以将某些不频繁访问的数据或历史数据通过持久化方式保存到磁盘上，从而释放出内存空间。

redis的哈希表相对于set的优势？

答：1) 具有高效的查找和访问性能；

2) 支持存储和访问复杂数据；

3) 可以灵活的增加和删除字段；

4) 存储空间占用小；

5) 支持批量操作

数据结构

说一个Java的一个排序方法？

答：冒泡排序：它是一种简单的交换排序算法，以升序为例，其核心思想是：1) 从第一个元素开始，比较相邻的两个元素。如果第一个比第二个大，则进行交换；

2) 轮到下一组相邻元素，执行同样的比较操作，再找下一组，直到没有相邻元素可比较为止，此时最后的元素应该是最大的数；

3) 除了每次排序得到的最后一个元素，对剩余元素重复以上步骤，直到没有任何一对元素需要比较为止。

用java实现的冒泡排序核心代码如下：

```
for(int i=0;i<arr.length-1;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```

时间复杂度：

最好的时间复杂度： $O(n)$ ，加入一个记录不需要继续排序的变量，当已排序时可达；

最差的时间复杂度： $O(n^2)$ ，一般情况；

平均时间复杂度： $O(n^2)$ ；

空间复杂度：每次递归须使用栈

最好、最差和平均空间复杂度： $O(1)$

拓展——其他排序方法

1) 快速排序：快速排序的思想很简单，就是先把待排序的数组拆分成左右两个区间，左边都比中间的基准数小，右边都比中间的基准数大。接着左右两边再做同样的操作，完成后再拆分再继续，一直到各区间只有一个数为止。

时间复杂度：

最优的时间复杂度： $O(n\log n)$ ，每一次取到的元素都刚好平分整个数组；最差的时间复杂度： $O(n^2)$ ，每一次取到的元素就是数组中最小/最大的，同冒泡排序；

平均时间复杂度： $O(n\log n)$ ；

空间复杂度：

每次递归须使用栈

最好的空间复杂度为： $O(\log n)$ ，每一次取到的元素都刚好平分整个数组；最差的空间复杂度为： $O(n)$ ，每一次取到的元素就是数组中最小/最大的，同冒泡排序

2) 直接插入排序：对于一个序列，选定一个下标，认为在这个下标之前的元素都是有序的，将下标所在的元素插入到其之前的序列中。接着再选取这个下标的后一个元素，继续重复操作，直到最后一个元素完成插入为止。我们一般从序列的第二个元素开始操作。

3) 希尔排序：把序列按下标的一定增量分组，对每组分别使用插入排序。随着增量减少，一直到一，算法阶数，整个序列变为有序。因此希尔排序又称为缩小增量排序。一般来说，初次取序列的一半为增量，以后每次减半，直到增量为一。

4) 简单选择排序：每一趟从未排序的区间找到一个最小元素，并放到第一位，直到全部区间有序为止。

5) 堆排序：根据初始数组构造大顶堆，每次交换第一个和最后一个元素，然后将除最后一个元素以外的其他元素重新调整为大顶堆。重复以上两个步骤，直到没有元素可以操作，就完成排序了。

6) 归并排序：将 n 个元素分成两个各含 $n/2$ 个元素的子序列；借助递归，两个子序列分别进行第一步操作，直到不可再分为止；此时每一层递归都有两个子序列，再将其合并，作为一个有序的子序列返回上一层，再继续合并，全部完成之后得到的就是一个有序的序列。

7) 基数排序：将整数按位数切割成不同的数字，然后按每个位数分别比较。为此需要将所有待比较的数值统一为同样的数位长度，数位不足的数在高位补零。

说一个有序链表增加查询效率的方法

答：可以采用二分查找算法，以升序排列的有序链表为，步骤是：

1) 初始化两个指针，‘start’和‘end’；分别指向链表的头和尾；2) 使用start和end指针计算出中间节点的位置mid: $mid = (start + end) / 2$; 3) 将中间节点与目标值进行比较：

如果中间节点的值等于目标值，则找到了目标节点，返回结果；

如果中间节点的值小于目标值，则目标节点可能在左半部分，将end指针更新为mid-1，并重复上述步骤；

如果中间节点的值大于目标值，则目标节点可能在右半部分，将start指针更新为mid+1，并重复上述步骤；

4) 如果start>end，则表示没有找到目标节点，返回结果。

操作系统

定位LinuxCPU占用率高的进程？

答：使用top/ps命令：可以查看系统中当前运行的进程列表，并按照CPU占用率的高低进行排序；

项目

介绍下你的项目

答：我的项目是专门为餐饮企业定制的一款软件产品，包括系统管理后台和移动端应用两部分。其中系统管理后台主要提供给餐饮企业内部员工使用，可以对餐厅的菜品、套餐、订单等进行管理维护。移动端应用主要提供给消费者使用，可以在线浏览菜品、添加购物车、下单等。

本项目一共分为三个阶段实现，第一阶段实现基本需求，第二阶段进行移动端优化，第三阶段对系统进行优化、提高系统访问性能。

说说为什么项目使用springboot?

答：1) 可以快速整合第三方框架，无需配置文件；2) 内嵌了servlet容器，降低了对环境的要求；3) 大大减少了开发时间，并提高了生产力；4) 避免了编写大量的样板代码、注释和XML文件；5) 提供了很多插件来开发和测试。

说说项目为什么使用redis?

答：redis不仅性能高，还能提供很多友好的数据类型，可以很好的满足我们的业务需求。

说说项目为什么使用mybatis-plus?

答：mybatis-plus是一个mybatis的增强工具，在mybatis的基础上只做增强不做改变，为简化开发、提高效率而生。所谓增强，就是封装好了一些增删改查操作，不用再写mapper.xml，可以直接调用它的方法即可。

介绍一下登录方法处理逻辑

答：1) 将页面提交的密码password进行md5加密处理；2) 根据页面提交的用户名username查询数据库；3) 如果没有查询到则返回登录失败结果；4) 密码比对，如果不一致则返回登录失败结果；5) 查看员工状态，如果已禁用状态，则返回员工已禁用结果；6) 登录成功，将员工id存入Session并返回登录成功结果。

过滤器具体处理逻辑

答：1) 获取本次请求URL；2) 判断本次请求是否需要处理；3) 如果不需要处理，则直接放行；4) 判断登录状态，如果已经登录，则直接放行；5) 如果未登录则返回未登录的结果；

这个项目的意义

答：传统餐饮企业的日常运作还是靠人工管理。从原料入库到客人点单，再到结账基本上都由人工完成记录。这样做不仅耗费人力资源而且容易导致记录丢失或者重复等错误，造成管理水平低下。

瑞吉外卖系统不仅能提高管理效率，还能给管理者提供一个企业运作的信息平台，同时帮助提升企业的形象以及增强餐饮企业业务管理和人员管理信息化，使企业从原材料进货到客人消费都有计算机管理并记录，可随时掌握餐厅客人点单结账等情况，并降低人低成本、提高工作效率。大多数此行业的运营者可以因此得利，与此同时，消费者订餐也变得更加方便。

如何区分管理员和员工?

答：设计一个字段标明是管理员还是普通员工，比如level字段，0是管理员，1是员工。然后把当前登录的用户都放到session中。在页面登录时，直接取出

登录用户，然后判断level是0还是1。

redis用来干啥？

答：1) 用来做缓存，可以提高效率。

2) 其次，还可以用redis进行分片处理。根据用户地域转发订单，以提高数据传输效率和提供更好的用户体验。例如，可以将“订单详细表”中的订单信息

按照城市进行分片，可以让快递员更直接的获取订单，从而大大提高了物流效率。

3) 此外，Redis可以实现搜索功能，它可以把筛选条件作为键，映射用户需要搜索的相关信息，可以让搜索引擎更快、更准确的找到订单信息。

4) 另外，Redis还可以用作计算和统计系统，可以通过redis来实时统计系统的数据，如订单数量、订单金额等，可以帮助及时发现问题，并对商业发展有效的改善指导。

在项目中哪些数据用了redis缓存？

答：一些菜品数据，即经常需要查询且变动不是很频繁的数据，常称作热点数据。

3.技术架构：spring、springboot、Mybatis-plus、mysql、redis。

负责模块：菜品管理、套餐管理、员工管理等的开发。

主要工作：1) 负责项目后端框架搭建。使用springboot搭建框架、持久层使用mybatis-plus来简化开发，基本的增删改查直接调用框架提供的方法，实现常用公共字段的自动填充；

2) 后台管理相关功能的开发，实现菜品管理、套餐管理、员工管理等功能；3) 根据API接口文档，使用springboot和mybatis-plus完成对前端数据的接收、处理、返回，实现对数据库数据的读取和改写等功能。

4) 使用redis数据库，运用springcache框架，实现对用户登录信息以及用户查询过菜单数据的缓存，提升用户的访问速度；

5) 利用mysql自带的二进制日志功能，实现mysql主从复制，并将主库从库分别部署到两台虚拟机上，减轻单点数据库读写压力，避免发生单点故障。

6) 使用Nginx部署前端界面，并配置反向代理，实现前后端分离。

项目难点：

1) 将用户id从存入httpsession改为存入threadlocal,客户端同次请求下的操作可动态获取登录用户id;

2) 将验证码从保存在httpsession改为缓存在redis中，解决验证码设置过期时间问题；

3) 使用sharding-JDBC实现数据库读写分离，一主多从；

4) 使用springcache和redis实现缓存功能，将产品数据缓存在redis中，缓解mysql性能问题。

在瑞吉外卖开发过程中遇到的最大挑战是什么以及如何解决的？

答：在项目的开发过程中，遇到的最大挑战是处理高并发请求和保证系统的性能以及可扩展性。当用户增加并且同时有多个用户在下单和浏览菜单时，后端系统需要能够快速响应并处理大量的请求，同时保证良好的用户体验。

为了解决这个挑战，我们采取了以下几个措施：

1) 基础架构优化：包括选择高性能的服务器和数据库，合理分配和配置服务器资源，以提高系统的整体性能；

2) 缓存机制：引入缓存机制来减轻数据库的负载。对于频繁读取的数据，例如菜单信息，可以将其缓存在内存中，减少对数据库的查询次数，提高响应速度。

3) 异步处理机制：使用异步任务队列，将一些耗时的操作，如订单的支付和配

送任务，放入任务队列中进行处理，以避免阻塞主线程并提高并发处理能力；

4) 监控和日志：建立监控系统来实时监测系统的性能指标、错误率和异常情况。通过日志记录关键操作和异常情况，可以及时发现问题并进行故障排查；

瑞吉外卖项目怎么实现的前后端分离

答：1) 在工程结构上，前后端代码不再混合在同一个maven工程中，而是分为前端工程和后端工程；

2) 前后端分离后，不仅工程结构发生变化，后期项目上线部署时，与之前也不同：

之前：前后端代码都混合在一起，我们只需要将后端和前端的代码统一打包成jar包，直接运行就可以了；

现在：拆分为前后端分离的项目后，最终部署时，后端工程会打成一个jar包，运行在Tomcat中；前端工程的静态资源，会直接部署在Nginx中进行访问。

3) 开发具体流程：

定制接口：这里的接口不是我们之前在service,mapper层定义的interface；而是一个http的请求地址，主要就是去定义：请求路径、请求方式、请求参数、响应数据等内容。

前后端并行开发：依据定义好的接口信息，前端人员开发前端的代码，服务端人员开发服务端的接口；在开发中前后端都需要进行测试，后端需要通过对应的工具来进行接口的测试，前端需要根据接口定义的参数进行数据模拟测试；

联调测试：当前后端都开发完毕并且自测通过之后，就可以进行前后端的联调测试了，在这一阶段主要就是校验接口的参数格式；

前后端联调测试通过之后，就可以将项目部署到测试服务器，进行自动化测试了。

一般都会要求30分钟无任何操作的情况下用户自动登出，这个是怎么实现的？

答：在用户登录的时候生成一个唯一id并且设置过期时间，将它放在redis中，每次操作都更新这个键的过期时间，这个键过期的时候就执行登出方法！

AQS是什么

答：AQS是JAVA中的同步器，类似于锁，有两个动作：获取和释放；

AQS原理

答：通过维护一个volatile变量state表示同步状态，当state为0时表示没有线程占用同步状态，当state为1时表示有一个线程占用同步状态。当多个线程竞争同步状态时，只有一个线程可以成功占用同步状态，其余线程将加入到AQS的同步队列中等待。当占用同步状态的线程释放同步状态时，AQS会从同步队列中选择一个线程唤醒，使其重新尝试获取同步状态。

完此项目的收获？

答：1) 了解了企业项目整体开发流程；

2) 掌握springboot和主流集成框架实现项目的快速开发；3)

掌握外卖行业相关的业务流程和功能架构；

4) 了解项目架构的演进过程

5) 掌握需求分析的方法，和需求到设计、实现的转换过程；

6) 掌握mysql主从复制配置过程；

7) 掌握读写分离开发过程

技术亮点

答：(1) 使用Redis缓存

当用户数量较多时，系统访问量大，频繁的访问数据库，数据库压力大，系统的性能下降，用户体验感差。因此使用Redis对数据进行缓存，从而减小数据库的压力，在数据更新时删除缓存，从而保证数据库和缓存的一致性，同时有效提高系统的性能和访问速度。

(2) 使用MySQL主从复制，进行读写分离

读和写数据的所有压力全都由一台数据库承担，压力大，数据库服务器磁盘损坏则数据丢失，单点故障。使用MySQL进行主从复制，主库进行写操作，从库进行读操作，从而减轻数据库负担，增大系统承受能力，提高系统性能。本项目使用Sharding-JDBC在程序中实现读写分离。

(3) 前后端分离部署，使用Nginx进行反向代理及负载均衡

前端页面部署到Nginx服务器中，后端代码部署到后端服务器中，使用Nginx对后端服务器进行反向代理，使用户只需要访问Nginx服务器便可获得后端服务器的服务（便于后期扩展集群，提高系统并发量）。

主从复制

答：主从的含义是：所有的写操作在主库上，读操作在一个或者多个从库上，当然主库也可以承担读请求，而从库的数据则通过主库复制而来，MySQL自带主从复制的功能。

主从复制步骤：

master将改变记录到二进制日志中；

slave将master的二进制日志拷贝到它的中继日志；

slave重做中继日志中的事件，将改变应用到自己的数据库中；

数据库层面是主从复制，应用层是sharding-jdbc读写分离。读写分离指的是利用数据库主从技术，分散读多个库以支持高并发的读，而写只在master库上。数据库的主从技术只负责对数据进行复制和同步，而读写分离技术需要业务应用自身去实现。Sharding-Jdbc通过简单的开发。可以方便的实现读写分离技术。**缓存是否会满，缓存满了怎么办？**

答：1）给缓存服务，选择合适的缓存逐出算法，比如最常见的LRU。

（LRU：如果一个数据在最近一段时间没有被访问到，那么在将来它被访问的可能性也很低，当数据所占据的空间达到一定阈值时，这个最少被访问的数据将被淘汰掉。）

2）针对当前设置的容量，设置适当的警戒值，比如10G的缓存，当缓存数据达到8G的时候，就开始发出报警，提前排查问题或者扩容。

3）给一些没有必要长期保存的key，尽量设置过期时间。

缓存是否允许丢失？丢失了怎么办？

答：根据业务场景判断，是否允许丢失。如果不允许，就需要带持久化功能的缓存服务来支持，比如Redis或者Tair。更细节的话，可以根据业务对丢失时间的容忍度，还可以选择更具体的持久化策略，比如Redis的RDB或者AOF。

消息队列（MQ）

使用消息队列（MQ）中间件服务，MQ天生就是异步的。一些额外的任务，可能不需要这个系统来处理，但是需要其他系统来处理。这个时候可以先把它封装成一个消息，扔到消息队列里面，通过消息中间件的可靠性保证把消息投递到关心它的系统，然后让其他系统来做相应的处理。

怎么解决超卖？

答：超卖问题是指在多线程环境下，当多个线程同时读取和修改共享资源时，可能会导致资源的重复分配或丢失。在电商平台等高并发场景中，超卖问题是一种常见的难题。本文将介绍如何使用Java来解决超卖问题，并通过一个示例来说明。

超卖问题通常出现在商品限量销售的场景中。假设有100个商品，多个用户同时下单购买，如果不加以控制，可能会导致超卖现象，即超过100个商品被售出。

超卖问题的主要原因是多线程环境下的竞争条件。当多个线程同时读取商品库存数量，并进行减少操作时，由于线程切换的不确定性，可能会导致多个线程都读取到相同的库存数量，从而导致库存减为负数的情况。

方法一：使用synchronized关键字加锁

使用synchronized关键字可以给方法或代码块加锁，确保同一时间只有一个线程可以执行这段代码。在解决超卖问题时，可以将减库存的操作使用synchronized关键字进行加锁。

方法二：使用ReentrantLock类

ReentrantLock是Java提供的可重入锁，与synchronized相比，它提供了更多灵活的操作，如可中断、超时等待和公平锁等。在解决超卖问题时，也可以使用ReentrantLock类来保证线程安全。

redis常见的数据结构？

答：redis支持五种数据结构，分别是string、list、set、hash、zset，即字符串、列表、集合、哈希、有序集合。

String：存储图片；

List：消息队列

Set：用来做统计；

Zset：可以用来做排行榜

数据结构及算法

常见的排序算法

答：1) 冒泡排序：

最坏的时间复杂度： $O(n^2)$;

平均时间复杂度： $O(n^2)$;

最好时间复杂度： $O(n)$;

空间复杂度： $O(1)$

2) 选择排序：每次从未排序的部分选择最大的元素，并将其放到已排序部分的末尾；

最坏的时间复杂度： $O(n^2)$;

平均时间复杂度： $O(n^2)$;

最好时间复杂度： $O(n^2)$;

空间复杂度: $O(1)$

3) 快速排序: 选择一个基准元素, 将比它小的元素放在左边, 比它大的元素放在右边, 再对左右两个部分递归地进行快速排序。

最坏的时间复杂度: $O(n^2)$;

平均时间复杂度: $O(n \log n)$;

最好时间复杂度: $O(n \log n)$;

空间复杂度: 取决于实现方式, 一般为 $O(n \log n) \sim O(n)$